Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Memory Management and Protection **3**

1.1 Context and Goals . . . 3
1.2 Where You Start . . . 3
1.2.1 The DEMO Switch . . . 4
1.3 Unit 1: Memory Management – the Ideas . . . 4
1.3.1 Why an Operating System Manages Memory . . . 4
1.3.2 What the Starting Code Does . . . 5
1.3.3 Static versus Dynamic Allocation . . . 6
1.3.4 The Memory-Management Spectrum . . . 6
1.3.5 A Rogue Task . . . 7
1.3.6 Paging: Pages, Frames and the MMU . . . 7
1.3.7 The TLB and the Cost of Paging . . . 9
1.3.8 Page Faults and Virtual Memory . . . 10
1.3.9 MPU versus MMU . . . 11
1.3.10 A Bad Pointer . . . 12
1.3.11 End of Unit 1 . . . 13
1.4 Unit 2: Task Partitions . . . 13
1.4.1 The Target: One Slice of RAM per Task . . . 13
1.4.2 Linker Tools You Need . . . 14
1.4.3 The Partition Block . . . 15
1.4.4 Checking Where Everything Landed . . . 16
1.4.5 `partition_init`: Giving the Variables Their Values . . . 16
1.4.6 Two Stack Pointers: MSP and PSP . . . 17
1.4.7 Starting a Task: `task_launch` and the Fake Frame . . . 17
1.4.8 SVC_Handler and SVC_dispatch . . . 19
1.4.9 How a Task Ends: `task_exit` and `sys_exit` . . . 20
1.4.10 The Two Stacks in gdb . . . 21
1.4.11 The Rogue Task, Partitioned . . . 21
1.4.12 End of Unit 2 . . . 21
1.5 Unit 3: The MPU . . . 22
1.5.1 The Cortex-M3 MPU: Regions . . . 22
1.5.2 Permissions: AP, XN and PRIVDEFENA . . . 23
1.5.3 The MPU Registers . . . 23
1.5.4 This Week's Two Regions . . . 23
1.5.5 Computing SIZE, RBAR and RASR . . . 24
1.5.6 Helpers and Prepared Calls . . . 25
1.5.7 Writing `mpu_init` and `mpu_set_task_region` . . . 25
1.5.8 Catching Violations: The MemManage Fault . . . 26

1.5.9 Running with the MPU On: Protection in Action 27
1.5.10 Why the Kernel Must Validate Pointers (`DEMO=badptr`) 28
1.5.11 How a Task Ends: `task_exit_current` 29
1.5.12 End of Unit 3 30
1.6 Where to Find the Code 31
1.7 A prova di coglione 31
1.7.1 Unità 1 31
1.7.2 Unità 2 34
1.7.3 Unità 3 38

Chapter 1

Memory Management and Protection

1.1 Context and Goals

At the end of Week 3 the kernel was small but working: unprivileged code reached the hardware only through system calls (`svc`). **Memory**, however, was completely open. The kernel's variables, the application's variables and every stack lived side by side in the same RAM, and unprivileged mode, which blocks certain instructions and the system registers, does not check ordinary loads and stores. The question of this week is therefore: **who may touch which memory?** The answer comes in three steps:

1. **Unit 1: Memory management – the ideas.** Why an operating system manages memory, static versus dynamic allocation, and the spectrum from flat memory to paging and virtual memory (page tables, TLB, page faults, effective access time), ending with MMU versus MPU. No code: you observe the starting code and see what it fails to protect.

2. **Unit 2: Task partitions.** Each task gets its own 4 KB slice of RAM, placed by the linker, with its stack at the bottom. The kernel starts a task on its own **Process Stack Pointer (PSP)** and ends it cleanly.

3. **Unit 3: The MPU.** You program the Cortex-M3 **Memory Protection Unit**, so that the hardware stops any task that accesses memory outside its partition, and the kernel checks every pointer that a task passes to it.

1.2 Where You Start

The code is in two sibling directories, `week-04/base/` and `week-04/solution/`, each with its own `Makefile`. You start in `base/` and turn it step by step into `solution/`. Unless otherwise specified, all paths in these notes are relative to `week-04/solution/`; QEMU runs with `-nographic` and is stopped with `Ctrl-C`.

Unlike Week 3, the base **builds and runs from the first minute**. It already runs two unprivileged tasks, Task A and then Task B, each with its own stack, but without any partitioning or protection. The long parts that are not the focus of this week are already prepared:

- the task table and the code that starts and ends a task (`kernel/task.c`);
- an `SVC_Handler` that finds the system-call frame on either the MSP or the PSP, and the new system call `sys_exit`;
- the CMSIS definitions of the MPU registers and of `MMFAR` (`include/cmsdk.h`);
- the two applications (`src/task_a.c`, `src/task_b.c`) and the `DEMO` switch of the `Makefile`.

What you write is about sixty lines: in **Unit 2** the partition block of the linker script (`scripts/mps2_m3.ld`); in **Unit 3** the two MPU functions (`kernel/mpu.c`), the address report in `MemManage_Handler` (`kernel/faults.c`) and the pointer check in `sys_write_impl` (`kernel/syscall.c`).

1.2.1 The DEMO Switch

`make run DEMO=`*name* builds Task B with a block of deliberate misbehavior, included through a preprocessor test such as `#if defined(DEMO_ROGUE)` in `src/task_b.c`. Each demo is built in its own directory (for example `build/demo-rogue/`), so switching between demos never needs a `make clean`; a plain `make run` builds the normal program in `build/`. The three demos are:

- `rogue`: B writes directly into the kernel's uptime counter;
- `badptr`: B passes a kernel address to `sys_write`;
- `overflow`: B recurses until its stack grows out of its own memory.

Running them on the base, after the partitions and after the MPU shows what each layer of protection can and cannot prevent. Note that a **correct** program prints the same output at every stage: what changes is what a **wrong** or malicious program is allowed to do.

1.3 Unit 1: Memory Management – the Ideas

1.3.1 Why an Operating System Manages Memory

Memory is one shared resource that several programs want at once: the kernel and every task. An operating system has four jobs with respect to memory:

- **Placement.** Decide where each program's code, data and stack go, so that they do not overlap. Two programs that both think they own `0x20000004` would destroy each other's data.
- **Sharing.** Let programs use the same memory when they should: one copy of a library serves every program, a buffer can be handed to a device, two programs can exchange data through a common area.
- **Protection.** Stop a program from reading or writing memory that is not its own, by mistake (a bad pointer, a stack that grows too far) or on purpose.

- **More programs than memory.** When the programs together need more than the RAM available, keep in RAM only the parts in use and the rest somewhere else.

In this unit you write no code. You look at the starting code and see which jobs it does. The short answer: only the first, and only partly.

1.3.2 What the Starting Code Does

Placement. In `week-04/base/`, build the program and ask binutils where the linker put four variables: `uptime_ms` and `tx_buf` (kernel), `a_lines` (Task A) and `b_seconds` (Task B):

```
make
arm-none-eabi-nm -n build/scratch-os.axf | grep -E '\b(uptime_ms|tx_buf|a_
```

```
20000004 B uptime_ms
20000008 b tx_buf
20000054 b a_lines
20000058 b b_seconds
```

The letter is the symbol type: `B`/`b` means `.bss` (upper case for global, lower case for `static`). The space before the names and the `$` after them keep only whole names; otherwise `systick_uptime_ms` would match too. All four variables are packed together just above `0x20000000`, and there is **no border** between kernel and task data: the linker placed them file after file, with the generic `*(.bss*)` rule. If Task B computed `0x20000054` and wrote there, it would change A's variable and nobody would notice. The starting code does placement only, per variable and not per task.

Behavior. `make run` (watch about six seconds, then `Ctrl-C`) prints:

```
scratch-os: kernel started
starting task A
A: hello from task A
A: uptime 0 ms (line 1)
A: uptime 1000 ms (line 2)
A: uptime 2000 ms (line 3)
task A exited
starting task B
B: hello from task B
B: uptime 4000 ms (second 1)
B: uptime 5000 ms (second 2)
B: uptime 6000 ms (second 3)
```

A runs to its end, then B starts: the two tasks **never run at the same time**. B's first line says `4000 ms` because the uptime is the kernel's clock since boot: B starts at about 3000 ms, after A's three waits, and prints one second later. The same output is printed by the finished program: a correct program does not change, only a wrong one does.

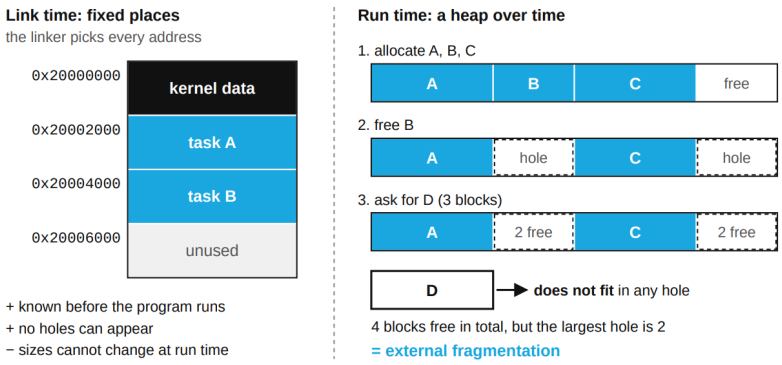
The board's memory. In QEMU's monitor, `info mtree -f` lists the memories. What the linker script calls FLASH is `mps.ssram1` at `0x00000000`, 4 MiB; what it

calls RAM is `mps.ssram23` at `0x20000000`, 4 MiB; both also appear at `+0x400000` (an **alias**: the same bytes at a second address). The list says every memory is just `ram`: nothing says who may access it. The bus answers any load or store from any code, privileged or not, so protection must come from somewhere else.

1.3.3 Static versus Dynamic Allocation

The other half of placement is **when** addresses are decided.

- **Static allocation** fixes every address and size at **link time**. The map file shows the whole layout before the program runs and no holes appear, because nothing is freed. The price is flexibility: sizes cannot change at run time.
- **Dynamic allocation** hands out memory at **run time** from a **heap** (`malloc`/`free`). It is flexible, but the layout depends on the order of the calls. After many allocations and frees, free memory is split into small **holes**: with 4 free blocks in total but no hole of 3, a request for 3 blocks **fails** although memory is free (**external fragmentation**).

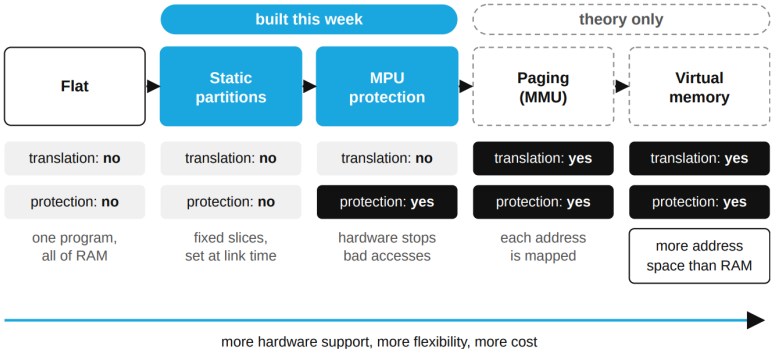


For an embedded system dynamic allocation has two more problems: it can **fail at run time**, where nobody can read the error, and the time of a `malloc` depends on how many blocks it must search, so it is **not constant**, which is bad for deadlines. This course allocates **statically**; every partition of Unit 2 is fixed at link time.

1.3.4 The Memory-Management Spectrum

Memory management methods can be lined up by asking two questions: is the address changed before it reaches memory (**translation**), and does the hardware stop forbidden accesses (**protection**)?

- **Flat.** Everything shares all of RAM, with no translation and no protection. The starting code is here.
- **Static partitions.** Each task gets a fixed slice of RAM at link time: better placement, but still no protection (Unit 2).
- **MPU.** Hardware checks every access against a short list of allowed regions: protection without translation (Unit 3).



- **Paging with an MMU.** Hardware maps every address to a different physical one: translation and protection together (theory only).
- **Virtual memory.** Paging plus the disk, so that programs can use more memory than the RAM has (theory only).

Each step needs more hardware and buys more flexibility.

1.3.5 A Rogue Task

This is what "no protection" means in practice. With `make run DEMO=rogue`, right after its greeting, Task B executes this block of `src/task_b.c`:

```
extern volatile uint32_t uptime_ms;      /* defined in another file; the l
printf( "B: writing to the kernel's uptime counter\n" );
uptime_ms = 0x12345678;                  /* a store into kernel memory */
```

```
B: writing to the kernel's uptime counter
B: uptime 305420896 ms (second 1)
B: uptime 305421896 ms (second 2)
```

`0x12345678` is 305 419 896 in decimal and the kernel keeps counting from there, so one second later B reads 305 420 896 ms: Task B has **changed the kernel's clock**, for everyone. Nothing stopped it, because unprivileged mode blocks only some **instructions** (for example `msr` to CONTROL) and the system registers, not **ordinary loads and stores** to RAM. The lesson: a bug in one task can silently break the kernel or another task, and the damage shows up far from its cause. To stop it, the hardware must check every access.

1.3.6 Paging: Pages, Frames and the MMU

Paging is used by large operating systems (Linux, Windows, macOS). You do not build it, but you need it to understand what a microcontroller does differently.

A program uses **virtual addresses** (VA); the memory chips see **physical addresses** (PA). Paging maps one to the other:

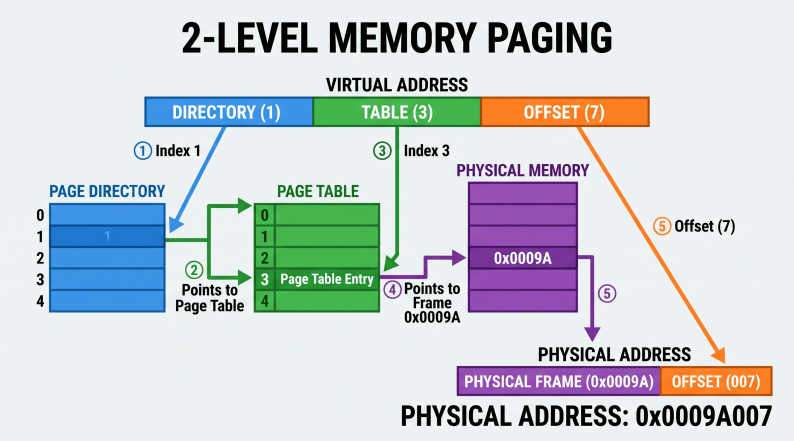
- The virtual space is cut into equal blocks called **pages** (often 4 KiB); physical memory is cut into **frames** of the same size. Any page can go in any frame.
- A **page table** says which frame holds each page, one table per **process**. Each entry has a **valid** bit (is the page in a frame at all?) and permission bits (read, write, execute, user or kernel only).

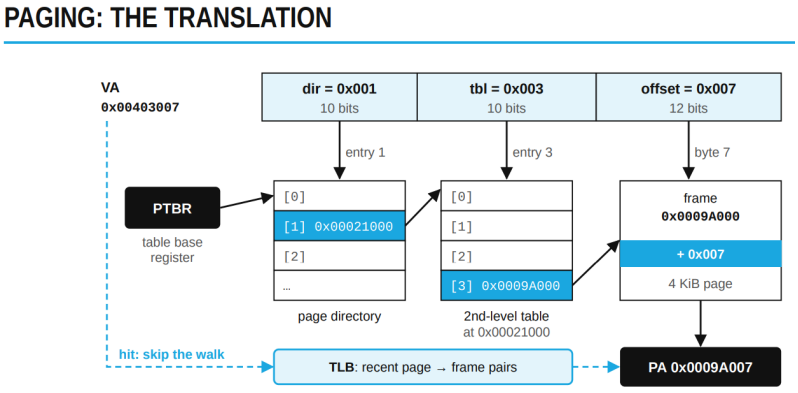
- The **MMU** (Memory Management Unit) is the hardware that translates **every** access and checks the permissions. It must be hardware: doing it in software for every load, store and fetch would be far too slow.

An address splits into a **page number** and an **offset**. Only the page number is translated; the offset is copied unchanged. With 4 KiB pages the offset is the low **12 bits** ($2^{12} = 4096$).

A two-level walk. With a 32-bit VA, one flat table indexed by 20 bits would need 2^{20} entries, that is 4 MiB **per process** even for a program using three pages. Real systems use **two levels**: a **page directory** whose entries point to second-level tables, with a register **PTBR** holding the directory address. The address is split into 10 bits of directory index, 10 bits of table index and 12 bits of offset, with 4-byte entries. Translate `VA = 0x00403007`:

```
0x00403007 = 0000000001 | 0000000011 | 000000000111
               dir = 1      tbl = 3      offset = 7
```





1. Read directory entry 1, at `PTBR + 1 × 4`; say it holds the second-level table at `0x00021000`.

2. Read entry 3 of that table, at `0x00021000 + 3 × 4 = 0x0002100C`; say it holds frame `0x0009A000`, valid.

3. PA = frame + offset = `0x0009A000 + 0x007 = 0x0009A007`.

Two levels pay off because a second-level table exists **only where memory is used**: a small process needs one directory (4 KiB) and a few tables instead of a 4 MiB table made mostly of invalid entries.

1.3.7 The TLB and the Cost of Paging

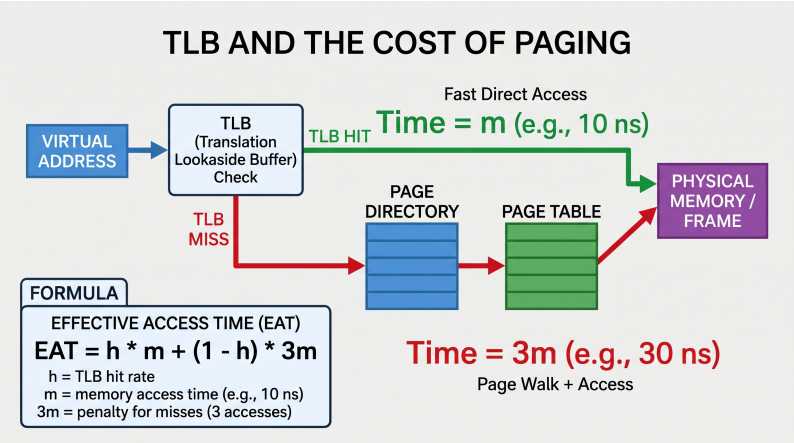
The walk costs **two extra memory reads** per access, which would make memory three times slower. The **TLB** (Translation Lookaside Buffer) is a small, fast cache inside the MMU that keeps recent translations (page number → frame, plus permissions). On a **hit** the frame comes straight from the TLB; on a **miss** the MMU walks the tables and stores the result. Programs reuse the same few pages (loop code, stack, current array), so most accesses hit. Since it is a cache, the OS must **flush** the stale entries when it changes a page table or switches process, otherwise a process could reach the previous one's frames.

Effective access time (EAT). It is the average time of one access, translation included. With `m` the time of one memory access and `h` the **hit ratio**, a hit costs `m` and, with a two-level table, a miss costs `3m` (directory read, table read, the access itself):

$$\text{EAT} = h * m + (1 - h) * 3m = m * (3 - 2h)$$

Each miss adds `2m`, so the **slowdown** over plain memory is $2 \times (1 - h)$: 1% of misses makes memory 2% slower. With `m = 10 ns` (a hit costs 10 ns, a miss 30 ns):

no TLB	EAT = 30 ns	3x slower
h = 0.80	EAT = 0.80 * 10 + 0.20 * 30 = 14.0 ns	+40%
h = 0.99	EAT = 0.99 * 10 + 0.01 * 30 = 10.2 ns	+2%



The misses, not the hits, set the cost, so `h` must be close to 1. Useful rules:

- **How high must `h` be?** For a slowdown of at most `s`: $2 \times (1 - h) \leq s$, so $h \geq 1 - s/2$ (at most 5% needs $h \geq 0.975$).
- **Deeper tables.** 64-bit systems use **four** levels: a miss costs `5m` and the slowdown is $4 \times (1 - h)$, twice as much for the same `h`.
- **Why `h` is high: locality.** A loop reading an array misses once per 4 KiB page and hits on the other 1023 words of that page.

- **TLB reach.** 64 entries of 4 KiB cover 64×4 KiB $=$ **256 KiB**; a program that jumps at random over 1 GiB misses almost always. Bigger pages (2 MiB) increase the reach.
- **After a flush** the TLB is empty and the first accesses miss: for a real-time task this is **jitter**, since access time depends on what the TLB happens to hold.

Exercise. With `m = 50 ns` and a two-level table: with no TLB the EAT is 150 ns (3x slower); with `h = 0.90` it is $0.90 \times 50 + 0.10 \times 150 = 60$ ns (+20%); with `h = 0.98` it is 52 ns (+4%); for a slowdown of at most 10% we need $h \geq 0.95$. With four levels and `h = 0.98` a miss costs 250 ns, so EAT $= 0.98 \times 50 + 0.02 \times 250 = 54$ ns (+8%). A loop that reads a 64 KiB array of 32-bit words once, with 4 KiB pages and an empty TLB, takes 16 misses out of 16 384 accesses, so $h = 1 - 16/16\,384 \approx 0.999$.

1.3.8 Page Faults and Virtual Memory

If the walk finds an entry with the valid bit clear, or the access breaks its permissions (a write to a read-only page, user code touching a kernel page), the MMU raises a **page fault**, an exception that records the address wanted:

1. If the address is not part of the program or the access is not allowed, the OS **kills the process**.
2. If the address is legal but the page is not in RAM, the OS finds a free frame, reads the page from disk, fills the table entry and sets the valid bit.
3. The handler returns and the hardware **restarts the faulting instruction**. It is restarted, not skipped, because its load or store never happened; restarting makes the fault invisible to the program.

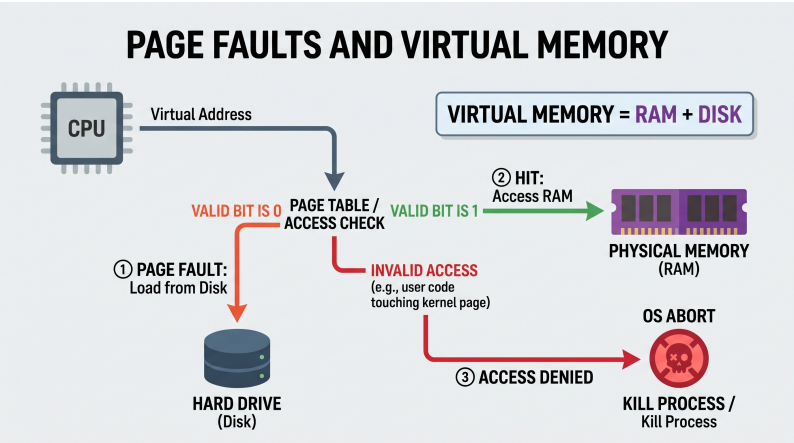


Figure 1.1: Page Hits and Faults

Cost. A fault that reads the disk costs **milliseconds**, against nanoseconds for an access. With `p` the page-fault rate, EAT $= (1 - p) \times m + p \times$ (service time). With `m = 200 ns` and a service time of 8 ms: $p = 1/1000$ gives about 8.2 μs, **forty times slower**; to lose at most 10%, $p \leq 2.5 \times 10^{-6}$, fewer than **one access in 400 000**. Virtual memory works only because locality makes faults so rare, and one

fault stops the program for milliseconds at an unpredictable moment: a real-time system cannot accept that.

Virtual memory is paging plus the disk, and it gives two things:

- **Isolation.** Each process has its own page table, so the same virtual address lands in different frames for different processes. A process cannot even **name** another process's memory: its own table translates every address into its own frames, or into a page fault.
- **More than RAM.** Pages not in use wait on disk and a page fault brings them back, so programs together can use more memory than the RAM has, as long as the pages in use at any moment fit.

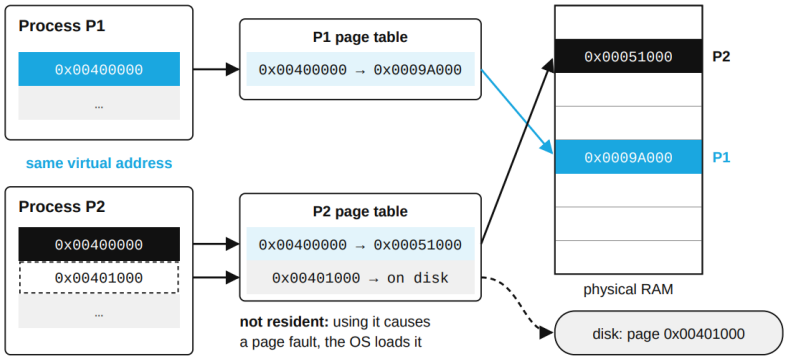


Figure 1.2: Memory Virtualitation

This answers the fourth job of the OS, and protection in the strongest way, but it needs an MMU, page tables in RAM and a disk.

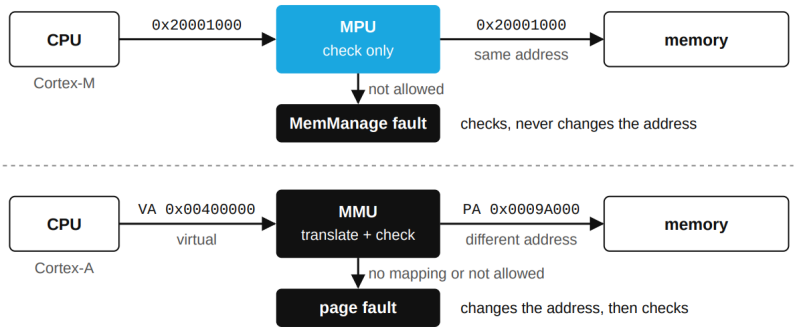
1.3.9 MPU versus MMU

A microcontroller has neither page tables nor a disk. The Cortex-M3 has an **MPU** (Memory Protection Unit): a short list of **regions**, each with a start address, a size and **permissions** (who may read, write, execute, separately for privileged and unprivileged code).

- On **every** load, store and fetch the MPU checks the address against the regions. If allowed, the access goes on at the **same** address; otherwise it is stopped (the store writes nothing, the load reads nothing) and the CPU raises a **MemManage** fault.
- There is **no translation**: every task sees real physical addresses, and the MPU only answers "may this code touch this address?"

Microcontroller cores (Cortex-M) have an MPU; application cores (Cortex-A, in phones and Linux boards) have an MMU, which turns VA into PA or raises a page fault. The MPU of this board is **off** after reset, which is why the rogue write went through; you turn it on in Unit 3.

Why does a microcontroller not have an MMU?



- **Cost and area:** an MMU needs page tables in RAM, a hardware walker and a TLB; an MPU is a handful of registers.
- **Power:** no walks and no extra reads on a miss.
- **Determinism:** a TLB miss makes an access slower by an unpredictable amount; an MPU check takes the same time every time, which matters for deadlines.
- **No disk** to swap to, and **small, fixed programs** for which regions decided at build time are enough.

What you give up is translation: every task must be placed at its own real address by the linker (Unit 2), and the MPU has only a few regions with rules on size and alignment (Unit 3).

1.3.10 A Bad Pointer

The last demo shows a different leak. With `DEMO=badptr`, Task B asks the **kernel** to touch kernel memory, by passing a kernel address to the system call of Week 3 ("send these `len` bytes from `buf` to the console"):

```
int r = sys_write( (const char *) &uptime_ms, 16 );
printf( "\nB: sys_write returned %d\n", r );
```

```
B: hello from task B
^K^@^@line 2)
A: u
B: sys_write returned 16
```

The 16 bytes start at `uptime_ms`: its 4 bytes hold `0x00000BB8` (3000 ms), stored low byte first (`B8 0B 00 00`), not printable, so the terminal shows odd symbols (`^K^@^@`). The next 12 bytes are the start of `tx_buf`, still holding old text of Task A. Task B has read the kernel's private data **through the kernel**: `sys_write` returns 16 because the kernel, running in the SVC handler, **privileged**, reaches any address it is given.

The consequence for Unit 3: an MPU that restricts only unprivileged code would **not** stop this leak, because the reads are the kernel's. The kernel must check the pointer itself before using it.

1.3.11 End of Unit 1

- An OS manages memory to **place** it, **share** it, **protect** it, and to run more programs than fit. The starting code does placement only, per variable, with no border between kernel and tasks.
- This course allocates **statically**, at link time: dynamic allocation fragments the heap, can fail at run time and takes variable time.
- **Paging:** the MMU translates page to frame through page tables, the TLB caches translations, a page fault restarts the instruction once the page is in. **Virtual memory** adds isolation and the disk.
- **Cost of paging:** EAT $= h \times m + (1 - h) \times 3m$ with two levels; `h` must be close to 1 and page faults extremely rare.
- An **MPU** checks every access against regions, with no translation: cheap, low-power, predictable. The Cortex-M3 has one, switched off. Without it, `DEMO=rogue` changes the kernel's clock and `DEMO=badptr` leaks kernel bytes through a system call.

1.4 Unit 2: Task Partitions

1.4.1 The Target: One Slice of RAM per Task

A **partition** is a slice of RAM that belongs to a single task and holds only its stack and variables. FLASH holds the code, the read-only data and the initial values of the task variables. The goal is this RAM layout:

```
0x20000000   Task A partition (4 KB): [ stack 1 KB | A's variables | padd
0x20001000   Task B partition (4 KB): [ stack 1 KB | B's variables | padd
0x20002000   kernel .data and .bss
   ...       (free RAM)
   top       kernel stack
```

The partitions follow five rules:

- **Fixed.** The linker chooses them at link time (static allocation of Unit 1): no task is created, grown or moved at run time.
- **4 KB, aligned to 4 KB.** The size is a power of two and each start is a multiple of 4096 (`0x20000000`, `0x20001000`), which is exactly the shape the MPU of Unit 3 can protect. The price is waste: a task that needs 1.1 KB still gets 4 KB.
- **RAM only.** Code and read-only data (`.text`, `.rodata`, string literals) stay in FLASH, **shared** by the kernel and both tasks: one copy of `printf` serves everybody.
- **Stack at the bottom.** The stack takes the lowest 1 KB and the variables sit above it. Since the stack grows **down**, an overflow leaves the partition immediately, below its start, where Unit 3 will make it fault. With the stack

at the top, an overflow would silently overwrite the task's own variables, and no protection hardware could tell that write from a legitimate one.

- **The kernel keeps the rest:** its `.data` and `.bss` come after the partitions and its stack stays at the top of RAM.

1.4.2 Linker Tools You Need

An **output section** has two addresses: the **run address** (VMA), where the code uses it, and the **load address** (LMA), where its bytes are stored in the image. They are usually equal, but not for `.data`. The partition block uses:

```
.task_a_data : { ... } > RAM AT > FLASH          /* runs in RAM, initial
_task_a_data_load = LOADADDR(.task_a_data);      /* the FLASH address of
.task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : { ... } > RAM
*task_a.o(.data .data.* .bss .bss.* COMMON)      /* these input sections ,
```

- `> RAM AT > FLASH` (as for Week 2's `.data`): variables run in RAM but their initial bytes are stored in FLASH, which keeps its content without power. Somebody must copy them before use: `Reset_Handler` does it for the kernel; for a partition the kernel does it just before the task's first run.

- `(NOLOAD)` reserves addresses but stores **nothing** in the image: right for a stack or padding.

- `ALIGN(x, 4096)` rounds `x` **up** to a multiple of 4096; before the colon it sets the section's start. For the first partition we use `ALIGN(ORIGIN(RAM), 4096)` because right after `.text` the location counter `.` is still a FLASH address; for B, `.` is already in RAM, so `ALIGN(4096)` is enough.

- `. = start + 4096` jumps to the end of the partition, whatever was placed before.

- **File patterns.** `*task_a.o(...)` takes the variables of `src/task_a.c` only, so the source needs no special marking. `.bss` is put in the same loaded section as `.data`: its zeros are stored in FLASH too (4 bytes here), and a single copy loop initializes every task variable.

- **The first match wins.** Every input section is placed by the first rule that matches it, so the partition rules must come **before** the kernel's generic `*(.data*)` and `*(.bss*)`, otherwise the task variables would stay in kernel RAM.

- **Exported symbols and `ASSERT`** (Week 2). The prepared kernel already uses six symbols per task (`_task_X_part_start`, `_part_end`, `_stack_top`, `_data_start`, `_data_end`, `_data_load`, with `X` = `a` or `b`); your edit only **repoints** them. `ASSERT(cond, "msg")` fails the link if `cond` is false, so a task that grows too big is caught at build time.

1.4.3 The Partition Block

All edits are in `scripts/mps2_m3.ld` (in your copy, `week-04/base/scripts/`), three in total:

1. After the line `TASK_STACK_SIZE = 0x400;` add `PART_SIZE = 0x1000;` (4 KB per partition).

2. Near the end, **delete** everything from the comment `/* >> UNIT 2: delete from this comment down to _task_b_data_load. <<` down to the line `_task_b_data_load = LOADADDR(.data);` included: the flat `.task_stacks` section and the empty data symbols. Keep `__StackLimit` and the `ASSERT` after it, which protect the kernel stack.

3. Replace the comment `/* >> UNIT 2: paste the task partitions here, before .data (see the slides). << */`, right after `.text` and before `_sidata = LOADADDR(.data);`, with the partition of A followed by that of B.

Task A's partition is made of three output sections:

```
.task_a_stack ALIGN(ORIGIN(RAM), 4096) (NOLOAD) : {
    _task_a_part_start = .;              /* partition start = stack bot
    . = . + TASK_STACK_SIZE;             /* reserve the 1 KB stack */
    _task_a_stack_top = .;               /* the stack grows down from h
} > RAM
.task_a_data : {
    . = ALIGN(4);
    _task_a_data_start = .;              /* run address (RAM) */
    *task_a.o(.data .data.* .bss .bss.* COMMON)
    . = ALIGN(4);
    _task_a_data_end = .;
} > RAM AT > FLASH
_task_a_data_load = LOADADDR(.task_a_data);
.task_a_pad (NOLOAD) : {
    . = _task_a_part_start + PART_SIZE;    /* jump to the partition en
    _task_a_part_end = .;
} > RAM
```

`.task_a_stack` starts at the first 4 KB boundary, `0x20000000`, reserves 1 KB and marks the stack top at `0x20000400`. `.task_a_data` collects the variables of `task_a.o`; the two `ALIGN(4)` keep both ends on a word boundary, because the copy loop copies whole 32-bit words. `.task_a_pad` only jumps the location counter to the end of the 4 KB, so the next section starts after A's full partition.

Task B's partition goes right after `.task_a_pad`, still before `_sidata`. It is **identical** with `a` replaced by `b` and with `ALIGN(4096)` instead of `ALIGN(ORIGIN(RAM), 4096)`, followed by the two link-time guards:

```
ASSERT(_task_a_data_end <= _task_a_part_end, "task A data does not fi
ASSERT(_task_b_data_end <= _task_b_part_end, "task B data does not fi
```

1.4.4 Checking Where Everything Landed

```
make
arm-none-eabi-nm -n build/scratch-os.axf | grep -E '_(_task_|a_lines|b_se

00000dd0 A _task_a_data_load
20000000 B _task_a_part_start
20000400 B _task_a_stack_top
20000400 d a_lines
20001000 B _task_a_part_end
20001000 B _task_b_part_start
20001400 B _task_b_stack_top
20001400 d b_seconds
20002000 B _task_b_part_end
20002004 B uptime_ms
```

(Output abridged: the `_data_start`/`_data_end` symbols and `_task_b_data_load` are omitted.) A's partition is `0x20000000`–`0x20001000` and B's is `0x20001000`–`0x20002000`: 4 KB each, on a 4 KB boundary, with the stack in the bottom 1 KB. `uptime_ms` moved from `0x20000004` to `0x20002004`, because the kernel's data now follow the partitions.

`a_lines` and `_task_a_stack_top` share the address `0x20000400`, but this is not an overlap: the stack top is an **exclusive** bound. The Cortex-M stack is *full descending* (a push first decrements `sp` by 4, then stores), so with `sp = 0x20000400` the first push writes at `0x200003FC` and `0x20000400` itself stays free for the first variable. The same holds at `0x20001400`. In the map file, the sections of `task_a.o` appear under `.task_a_data`: the generic `*(.data*)` found nothing left of that file, because the partition rule came first (**first match wins**).

1.4.5 `partition_init`: Giving the Variables Their Values

The task variables run in RAM but start out in FLASH, so someone must copy them. This is `partition_init` in `kernel/task.c`:

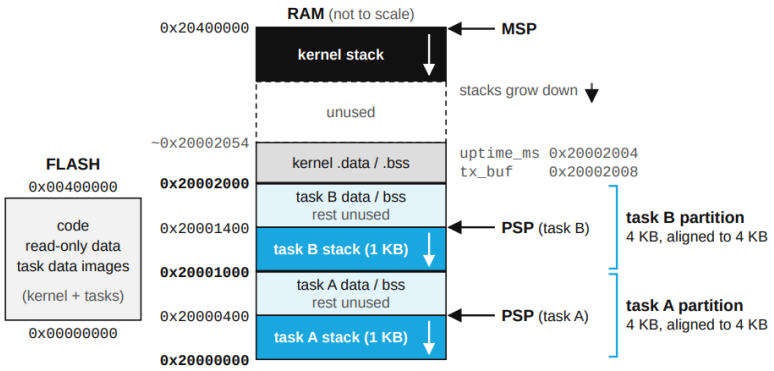
```
void partition_init( const struct task *t )
{
    const uint32_t *src = t->data_load;      /* initial values, in FLASH *
    uint32_t *dst = t->data_start;           /* the variables, in RAM */
    while ( dst < t->data_end )
        *dst++ = *src++;                     /* copy one word, move both
}
```

`struct task` (`kernel/task.h`) holds what the kernel knows about a task: name, entry function, stack top, partition bounds and the three data addresses; the table `tasks` fills it from the linker symbols, declared as arrays (for example `extern uint32_t _task_a_stack_top[];`) because only their address matters. It is the same loop as `Reset_Handler`'s copy of the kernel's `.data`, run once per task by `task_launch`. A task's `.bss` needs no zeroing loop, because its zeros are stored inside the loaded section. In the base the loop copied **nothing**: the flat layout set `_data_start` equal to `_data_end`, which let the prepared code run unchanged from the first minute.

1.4.6 Two Stack Pointers: MSP and PSP

The Cortex-M3 has two stack pointers; `sp` (`r13`) is the one in use, and `mrs`/`msr` can name each directly.

- **MSP** (main): always used in **handler mode**, and in thread mode after reset. It is the kernel's stack, at the top of RAM.
- **PSP** (process): used in thread mode when **SPSEL** (CONTROL bit 1) is 1. Each task's stack is a PSP stack inside its partition.
- **nPRIV** (CONTROL bit 0): when 1, thread mode is unprivileged. A task runs with **CONTROL = 3** (nPRIV = 1, SPSEL = 1).
- **EXC_RETURN** is the value in `lr` that ends an exception: `0xFFFFFFF9` returns to thread mode on MSP, `0xFFFFFFFD` to thread mode on PSP (the hardware sets SPSEL accordingly). On exception **entry** the hardware puts an EXC_RETURN in `lr`, and its **bit 2** tells which stack holds the pushed frame: 1 = PSP, 0 = MSP.



The stack must be **inside the partition**: it holds locals, return addresses and the exception frames. If it lived in kernel RAM, you could not restrict a task to its own 4 KB without taking its stack away. And since handler mode always uses MSP, the kernel's work (interrupts, system calls) never runs on a task's stack.

1.4.7 Starting a Task: `task_launch` and the Fake Frame

Why use a Fake Stack Frame? The kernel cannot simply launch a user task using a standard function call. Because the kernel operates in **Handler Mode** (using the Main Stack Pointer, MSP, with full privileges), a standard call would cause the task to inherit this exact state. This would destroy system isolation, as a bug in the user task could silently corrupt kernel memory or crash the entire system.

To safely execute, a user task must run in a strictly confined environment:

1. In **Thread Mode** (normal execution state).
2. **Unprivileged** (restricted access to system registers and memory).
3. On its own **Process Stack Pointer (PSP)**.

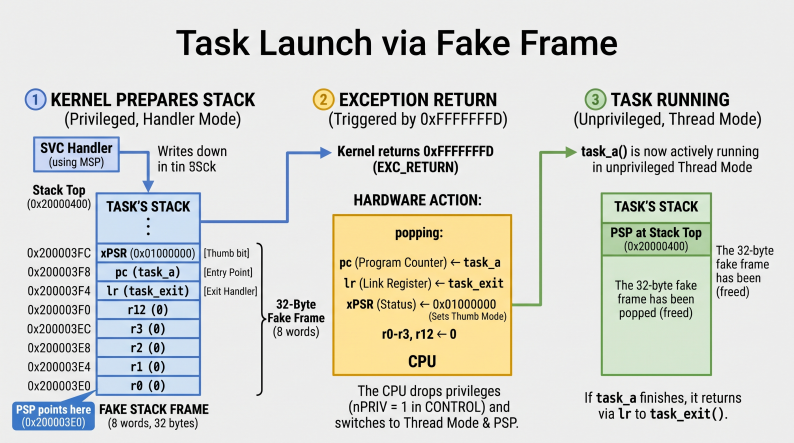
On the ARM Cortex-M architecture, there is no single instruction to simultaneously change the stack pointer, drop privileges, switch execution modes, and jump to a new address. The *only* hardware mechanism capable of performing this critical "triple jump" safely and atomically is an **Exception Return**.

Since a newly created task has never actually been interrupted, the kernel must trick the hardware by **faking an interrupt context**:

- **Faking the past:** The kernel manually crafts an 8-word (32-byte) frame on the task's stack. This contains the exact layout of data the hardware would expect to find if the task had been previously running and then interrupted (including the starting `pc` and the `xPSR` status register).

- **Triggering the mechanism:** The kernel points the PSP to this newly created frame and commands the CPU to return from the current interrupt using the magic code `0xFFFFFFFD`.

- **The hardware restores:** The CPU blindly accepts the fake frame. It pops the registers from the PSP, drops its privileges, switches to Thread Mode, and jumps to the task's starting address, successfully launching it in the exact restricted conditions required by the Operating System.



The three steps of `task_launch` The boot process starts from `main` with the `svc #5` system call. The handler intercepts the call and launches `task_launch()`, which performs three operations:

1. **Bookkeeping:** records which task is starting, prepares the MPU to protect its partition, and initializes its RAM variables (`partition_init()`).

2. **Manual Frame Creation:** allocates 8 words (32 bytes) just below the task's stack top to set up the initial CPU context.

Index	Register	Value and Purpose
`frame[0..4]`	`r0-r3, r12`	`0` (unused at startup).
`frame[5]`	`lr` (Link)	`task_exit`: if the task function returns, it will jump here to exit cleanly instead of crashing.
`frame[6]`	`pc` (Program)	The task's entry point address (e.g., `task_a`).
`frame[7]`	`xPSR`	`0x01000000`: sets the **Thumb bit**. Mandatory on Cortex-M processors; without it, a crash (`UsageFault`) occurs.

3. **Context Switch:** through an Assembly block, it loads the frame address into the `PSP` register, sets the `nPRIV` bit in the `CONTROL` system register (to drop privileges), and forces the return of the value `0xFFFFFFFD` (`EXC_RETURN_THREAD_PSP`).

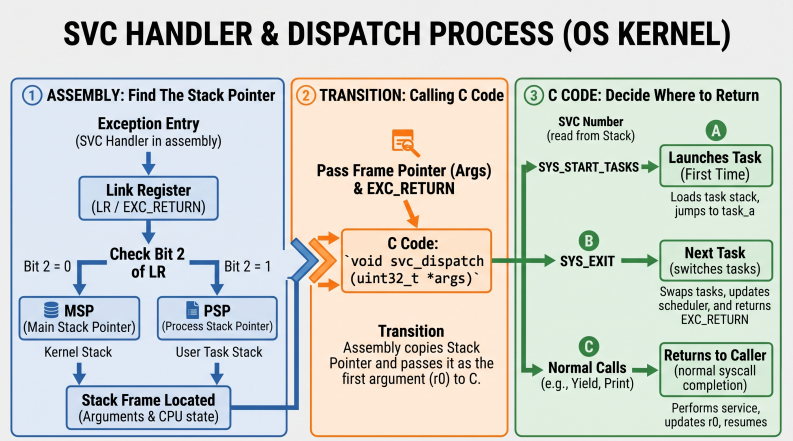
The final result When the handler returns `0xFFFFFFFD`, the hardware "falls for" the fake exception: it pops the 32 bytes, restoring the registers as dictated by the frame. Execution thus jumps to the task's `pc` (Thread mode, unprivileged), while the `PSP`, freed from the frame, sits exactly at the top of the stack, ready for standard program execution.

1.4.8 SVC_Handler and SVC_dispatch

In the basic version of our OS, system calls (`svc`) always returned to the exact same place they came from. Now, things are more complex because an `svc` can be triggered from two different environments:

1. From **main()**: just once, to start the OS. It uses the **MSP** (Main Stack Pointer).

2. From **tasks**: for standard system calls. They use the **PSP** (Process Stack Pointer).



1. Finding the right Stack (Assembly) When the handler starts, it needs to find the saved CPU state (the frame). To know which stack was active, the hardware leaves a clue in the **Link Register (lr)** called the `EXC_RETURN` code. By checking **bit 2** of this code, the handler knows where to look:

- `0xFFFFFFF9` (bit 2 is **0**): The frame is on the **MSP**.
- `0xFFFFFFFD` (bit 2 is **1**): The frame is on the **PSP**.

Here is how the assembly checks this and calls the C dispatcher:

```
tst lr, #4          /* Check bit 2 of LR (LR AND 4) */
ite eq              /* If-Then-Else based on the result */
mrseq r0, msp       /* IF bit 2 == 0: r0 = MSP */
mrsne r0, psp       /* ELSE bit 2 == 1: r0 = PSP */

mov r1, lr          /* r1 = exc_return (save LR before calling C) */
bl svc_dispatch     /* Call C: r0 = svc_dispatch(r0=frame, r1=exc_retur
bx r0               /* Return from exception using the code C gave us i
```

2. Directing the Flow (C code) Previously, the assembly handler ended with a simple branch (`b`) to go back. However, calls like `SYS_START_TASKS` and `SYS_EXIT` must change the flow entirely (e.g., leaving `main` to enter a task).

To achieve this, `svc_dispatch` is now responsible for deciding **where to return**. It takes the frame and the original `EXC_RETURN` code as arguments, and **returns a new EXC_RETURN code** back to the assembly:

```
uint32_t svc_dispatch( uint32_t *frame, uint32_t exc_return )
```

Inside the C function, it handles three specific scenarios:

- `SYS_START_TASKS`: Instead of returning to `main`, it returns `task_start_first()`, returning the code needed to launch Task A.
- `SYS_EXIT`: The task is done. It returns `task_exit_current()`, giving the code to launch the next task or go back to `main`.
- **Normal calls (Read/Write/etc.):** The kernel simply writes the function's result into `frame[0]` (which becomes `r0` for the task) and returns the **original** `exc_return`. Execution resumes exactly where it left off.

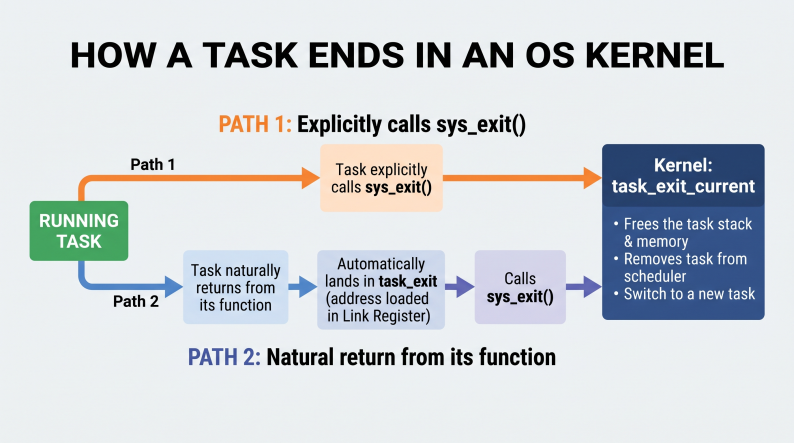
By doing this, the kernel respects a golden rule: **it never waits inside a handler**. Every system call ends immediately, smoothly guiding the CPU into whatever should run next.

1.4.9 How a Task Ends: `task_exit` and `sys_exit`

The user side is in `lib/syscalls.s`. `sys_exit` is a stub with the new number `SYS_EXIT` (4); since it never returns, it ends with a branch to itself instead of `bx lr`. `task_exit` is the address that `task_launch` puts in `frame[5]`, the task's `lr`:

```
sys_exit:
    svc #4
    b sys_exit
```

```
task_exit:
    bl sys_exit     /* does not come back
    b task_exit     /* never reached */
```



A task can therefore end by calling `sys_exit()` itself, or simply by returning from its function, landing in `task_exit`; Task A uses the second way. The kernel side, `task_exit_current`, is read in Unit 3.

1.4.10 The Two Stacks in gdb

With `make debug`, then `arm-none-eabi-gdb build/scratch-os.axf`, `target remote :1234`, `break *task_a`, `break *task_b`, `continue`, and `p/x $control`, `p/x $psp`, `p/x $msp`, `x/8wx $psp-32` at each stop (the `*` stops at the very first instruction, before any push), you measure:

- **CONTROL = 3** in both tasks (unprivileged, on PSP).
- **PSP** is `0x20000400` in A and `0x20001400` in B: the top of the bottom 1 KB of each partition, so inside its own partition.
- The popped **frame** still shows `lr = 0xb37` (`task_exit` plus 1, the Thumb bit), pc = `task_a` or `task_b`, and `xPSR = 0x01000000`.
- **MSP** is `0x203fffd8` in both: it holds `main`'s waiting frame, and tasks never touch MSP.

1.4.11 The Rogue Task, Partitioned

Running `make run DEMO=rogue` again (`make` rebuilds `build/demo-rogue/`), Task B still stores `0x12345678` into `uptime_ms`, now at `0x20002004`, outside B's partition, and the clock is corrupted **exactly as in Unit 1**. The linker only decided **where** things are: a store to `0x20002004` is an ordinary store, and nothing at run time compares it with B's partition `0x20001000`–`0x20002000`. Partitions **arrange** memory but do not **enforce** anything. What has changed is its shape: each task's memory is now one aligned 4 KB block, so one hardware rule per task is enough to describe what it may touch. That rule is Unit 3.

1.4.12 End of Unit 2

- A **partition** is a fixed slice of RAM for one task, 4 KB and aligned to 4 KB, with the stack at the bottom and the variables above it; code and read-only data stay shared in FLASH.

- The linker builds partitions with `ALIGN(..., 4096)`, `(NOLOAD)` for stack and pad, `> RAM AT > FLASH` for the variables and `*task_a.o(...)` to pick one file. The **first match wins**, so partition rules come before the kernel's, and `ASSERT` checks the size.

- `partition_init` copies each task's variables, zeros included, from FLASH before the first run.

- Tasks run unprivileged on **PSP** with **CONTROL = 3**; the kernel and `main` use **MSP**.

- `task_launch` writes an 8-word frame and returns `0xFFFFFFFD`, the only way into a task. `SVC_Handler` finds the frame with `tst lr, #4` and `svc_dispatch` returns the EXC_RETURN to use.

- `DEMO=rogue` still corrupts the kernel: the linker arranges memory but does not enforce anything.

1.5 Unit 3: The MPU

1.5.1 The Cortex-M3 MPU: Regions

Unit 1 introduced the MPU as a short list of regions checked on every access. On this CPU:

- There are **8 regions**, numbered 0 to 7 (`MPU_TYPE` bits 15:8 hold the count; on this board it reads `0x800`, that is 8).

- Each region has a **base address**, a **size** and **permissions**, and counts only when its ENABLE bit is set.

- The **size** is a power of two of at least 32 bytes, up to 4 GB.

- The **base must be a multiple of the size** (aligned to its size): a 4 KB region can start at `0x20000000` or `0x20001000`, not at `0x20000400`. This is why Unit 2 made each partition 4 KB and aligned to 4 KB: one region covers one partition exactly.

- When an **unprivileged** access falls in no enabled region, or its region refuses it, the access is **stopped** and the CPU raises a MemManage fault.

- For **privileged** code, the bit **PRIVDEFENA** chooses between the same rule and the board's default memory map wherever no region matches.

The MPU is a **core peripheral** at `0xE000ED90` and above, writable only by privileged code, so a task cannot switch it off.

1.5.2 Permissions: AP, XN and PRIVDEFENA

AP (access permissions) is a 3-bit field with one right for privileged and one for unprivileged code:

AP	Privileged	Unprivileged	Used here for
000	no access	no access	
001	read/write	no access	
010	read/write	read-only	
011	read/write	read/write	the running task's partition (MPU_AP_FULL_RW)
110	read-only	read-only	FLASH (MPU_AP_RO)

XN (execute never) forbids instruction **fetches** from the region (reads and writes follow AP). A partition holds data and stack, never code, so it gets XN: a task cannot run code it has written into RAM, which closes a classic attack.

PRIVDEFENA is bit 2 of MPU_CTRL. When 1, a **privileged** access that hits no region uses the default memory map, so the kernel keeps access to all RAM and peripherals without a region for each, and only the tasks are restricted. When 0, even the kernel may touch only what the regions allow. This week uses PRIVDEFENA = 1: the kernel is trusted and the MPU's purpose is to contain the tasks.

1.5.3 The MPU Registers

Register	Address	Fields used this week
MPU_TYPE	0xE000ED90	bits 15:8: number of regions (read-only)
MPU_CTRL	0xE000ED94	bit 0 ENABLE (the MPU checks accesses); bit 2 PRIVDEFENA
MPU_RNR	0xE000ED98	region number: which region RBAR/RASR refer to
MPU_RBAR	0xE000ED9C	bits 31:5 base; bit 4 VALID; bits 3:0 REGION
MPU_RASR	0xE000EDA0	bit 28 XN; bits 26:24 AP; bits 18:16 S, C, B; bits 5:1 SIZE; bit 0 ENABLE

There is one pair RBAR/RASR for all eight regions, and RNR selects the current one. As a shortcut, writing RBAR with **VALID** (bit 4) set also sets the region number from its REGION field, so a single write replaces RNR then RBAR; this week's code uses it.

The **SIZE** field (5 bits) holds $\log_2(\text{size}) - 1$, so a region has $2^{\text{SIZE}+1}$ bytes; the -1 lets 5 bits reach 4 GB. **S, C, B** (shareable, cacheable, bufferable) describe the memory type for caches and multi-core systems; this CPU has no cache, so they change nothing here (the code sets C for FLASH, S and C for RAM). TEX and SRD stay at 0.

1.5.4 This Week's Two Regions

- **Region 0** covers FLASH: base 0x00000000, size 4 MB, read-only and executable, the same for every task (code and string constants).

- **Region 1** covers the partition of the task running **now**, read/write and XN. The kernel moves it every time it starts a task.
- **Everything else** has **no region** for unprivileged code: the other task's partition, kernel data and stack, every peripheral. Any access by a task there causes a fault.

The "MPU state" of a task is therefore just one value: the base address of Region 1.

1.5.5 Computing SIZE, RBAR and RASR

SIZE $= \log_2(\text{size}) - 1$; since it sits in bits 5:1 it is multiplied by 2 before ENABLE (bit 0) is added:

- Partition: $4096 = 2^{12}$, SIZE = 11 (`0b01011`); $11 \times 2 =$ `0x16`, with ENABLE `0x17`.
- FLASH: 4 MB $= 2^{22}$, SIZE = 21 (`0b10101`); $21 \times 2 =$ `0x2A`, with ENABLE `0x2B`.

RBAR = base | VALID (`0x10`) | region number: Region 0 gives `0x00000010`; Region 1 gives `0x20000011` for Task A and `0x20001011` for Task B. The base fills bits 31:5 and a base aligned to a size of at least 32 always ends in five 0 bits, leaving room for VALID and REGION.

RASR of Region 1 for Task B is `0x13060017`:

Bits	Field	Value	Meaning
28	XN	1	no instruction fetch (`0x10000000`)
26:24	AP	`011`	read/write for both (`0x03000000`)
18, 17	S, C	1, 1	shareable, cacheable (`0x00060000`)
5:1	SIZE	11	2^{12} = 4 KB (`0x16`)
0	ENABLE	1	region on (`0x01`)

The sum is `0x10000000 + 0x03000000 + 0x00060000 + 0x16 + 0x01 = 0x13060017`. Region 0 is built the same way, from AP `110` (`0x06000000`), C (`0x00020000`) and `0x2B`, without XN: `0x0602002B`.

Three exercises:

- *A 32 KB region: SIZE and low byte of* ***`RASR`****?* 32 KB $= 2^{15}$, SIZE = 14, $14 \times 2 =$ `0x1C`, with ENABLE `0x1D`.
- *Could it start at* ***`0x20001000`****?* **No**: the base must be a multiple of 32 KB (`0x8000`); `0x20000000` and `0x20008000` are valid.
- *A task needs 5 KB: which region size?* **8 KB**, the next power of two (3 KB unused), the same price as for Unit 2's partitions.

1.5.6 Helpers and Prepared Calls

- `MPU_RASR_SIZE( n )` (`include/cmsdk.h`) takes $n = \log_2(\text{size})$ and computes `((n) - 1) « 1`: write `MPU_RASR_SIZE( 22 )` for 4 MB and `( 12 )` for 4 KB.
- `__builtin_ctz( x )` (GCC) counts the trailing 0 bits of `x`; for a power of two this is its $\log_2$ (`__builtin_ctz( 4096 )` is 12).
- `dsb` waits until every earlier write has really finished and `isb` then makes the following instructions see the new settings; after writing MPU registers you use both, so no later access is checked against the old regions.

`cmsdk.h` also gives the registers as a `struct` (`MPU->CTRL`, `MPU->RBAR`, `MPU->RASR`, ...) and one macro per field (`MPU_RBAR_VALID`, `MPU_RASR_XN`, `MPU_RASR_AP( v )` with `MPU_AP_RO` and `MPU_AP_FULL_RW`, `MPU_RASR_S`, `_C`, `_ENABLE`, `MPU_CTRL_ENABLE`, `MPU_CTRL_PRIVDEFENA`). The calls are prepared: `main` calls `mpu_init()` once before its `svc #5`, and `task_launch` calls `mpu_set_task_region( part_start, part_end - part_start )` before entering each task. In the base both bodies are empty, so the MPU is off.

1.5.7 Writing `mpu_init` and `mpu_set_task_region`

In `kernel/mpu.c` replace both functions, from `void mpu_init( void )` to the end of the file. `mpu_init` programs Region 0 and switches the MPU on:

```
void mpu_init( void )
{
    /* Region 0 = FLASH: base 0, 4 MB = 2^22 B, read-only, code may run.
    MPU->RBAR = 0x00000000 | MPU_RBAR_VALID | 0;          /* = 0x00000010
    MPU->RASR = MPU_RASR_AP( MPU_AP_RO ) | MPU_RASR_C
              | MPU_RASR_SIZE( 22 ) | MPU_RASR_ENABLE;    /* = 0x0602002B

    /* MPU on. PRIVDEFENA: the kernel keeps the default memory map. */
    MPU->CTRL = MPU_CTRL_ENABLE | MPU_CTRL_PRIVDEFENA;     /* = 5 */
    __asm volatile ( "dsb" ::: "memory" );   /* finish the writes ... */
    __asm volatile ( "isb" ::: "memory" );   /* ... before the next instr
}
```

After reset the MPU is disabled: until `ENABLE` is set in `MPU->CTRL`, no protection is enforced however the regions are programmed. Region 1 is not set yet, but this is not a problem: `main` and the kernel are privileged and keep the default map through `PRIVDEFENA`, and no unprivileged code runs before `task_launch` sets Region 1. The second function is called by `task_launch` every time a task starts, and moves Region 1 onto that task's partition:

```
void mpu_set_task_region( uint32_t base, uint32_t size )
{
    /* log2 of a power of two = its trailing zero bits: 4096 -> 12. */
    uint32_t log2_size = __builtin_ctz( size );

    /* Region 1 = the partition: read-write, never executable (XN). */
    MPU->RBAR = base | MPU_RBAR_VALID | 1;                 /* B: 0x2000101
```

```
    MPU->RASR = MPU_RASR_XN | MPU_RASR_AP( MPU_AP_FULL_RW )
              | MPU_RASR_S | MPU_RASR_C
              | MPU_RASR_SIZE( log2_size ) | MPU_RASR_ENABLE; /* 0x130600
    __asm volatile ( "dsb" ::: "memory" );
    __asm volatile ( "isb" ::: "memory" );
}
```

The function computes SIZE from the size it receives, which must be a power of two with `base` a multiple of it (the linker script guarantees both). The lines `(void) base;` and `(void) size;` of the base disappear: they only silenced "unused parameter" warnings while the body was empty. `make` now builds with no errors and no warnings.

1.5.8 Catching Violations: The MemManage Fault

When the MPU refuses an access, the access does **not** happen (a store writes nothing, a load reads nothing) and the CPU raises **MemManage**, exception 4, enabled at boot by `MEMFAULTENA` in `SHCSR` (Week 2). `MemManage_Handler` in `kernel/faults.c` runs privileged on the MSP. The low byte of **CFSR** (`0xE000ED28`) tells why:

Bit	Name	Set when
0	`IACCVIOL`	an **instruction fetch** was refused (for example from an XN region)
1	`DACCVIOL`	a **data** load or store was refused
3	`MUNSTKERR`	popping a frame on **exception return** was refused
4	`MSTKERR`	pushing the 8-word frame on **exception entry** (stacking) was refused
7	`MMARVALID`	**MMFAR** holds the refused address

MMFAR (`0xE000ED34`, `SCB->MMFAR`) holds the refused data address. Read it only if `MMARVALID` is 1: for a fetch or stacking fault the CPU does not write it, and it may hold a stale value. Several bits can be set at once.

Reporting the address. In `MemManage_Handler`, replace the line `kprintf( "MemManage: CFSR=0x%x\n", SCB->CFSR );` and the comment `/* >> UNIT 3: report MMFAR and the cause bits here. << */` with the code below (the `halt();` that follows stays):

```
    uint32_t cfsr = SCB->CFSR;
    kprintf( "MemManage: CFSR=0x%x\n", cfsr );

    /* MMFAR holds a real address only when MMARVALID is set. */
    if ( cfsr & SCB_CFSR_MMARVALID )
        kprintf( "  address 0x%x\n", SCB->MMFAR );

    /* Name the cause. Several bits can be set at once. */
    if ( cfsr & SCB_CFSR_DACCVIOL )
        kprintf( "  cause: data access\n" );
    if ( cfsr & SCB_CFSR_IACCVIOL )
        kprintf( "  cause: instruction fetch\n" );
```

```
if ( cfsr & SCB_CFSR_MSTKERR )
    kprintf( "  cause: stacking\n" );
```

`CFSR` is read once so that every test sees the same value, and the cause tests are separate `if`s rather than an `else if` chain because several bits can be set together. The masks `SCB_CFSR_...` are in `include/cmsdk.h`. `kprintf` works inside the handler because the handler is privileged.

1.5.9 Running with the MPU On: Protection in Action

Once the MPU is active, it strictly enforces the memory boundaries set by the linker. Here is how it affects different execution scenarios:

1. The Normal Run (`make run`) The system behaves exactly as in previous units, but now under hardware supervision. Tasks A and B run correctly because the MPU permits access to only two areas:

- **Region 0 (FLASH):** Where the code and string constants reside.
- **Region 1 (Task Partition):** Where the task's variables, stack, and local buffers (like the `printf` buffer) live.

When a task calls `sys_write` or `sys_uptime`, the context switches to the kernel (Handler Mode). The kernel is privileged and bypasses the MPU's user restrictions, freely accessing the UART and kernel data. As the OS switches from Task A to Task B, the MPU's Region 1 is simply updated on the fly to point to B's partition instead of A's.

2. The Rogue Task (`DEMO=rogue`) If Task B maliciously (or accidentally) attempts to overwrite a kernel variable like `uptime_ms`, the MPU intervenes instantly. The store instruction **never executes**. Instead, the hardware halts the task and triggers a **MemManage Fault**:

```
B: writing to the kernel's uptime counter
MemManage: CFSR=0x82
  address 0x20002004
  cause: data access
```

- `0x82` **(DACCVIOL + MMARVALID):** Means a data access was denied, and the faulting address is securely recorded.
- `0x20002004`**:** The exact address of `uptime_ms`.

The clock remains uncorrupted. The bug is caught exactly at the instruction that caused it, rather than causing a mysterious failure later.

3. The Stack Overflow (`DEMO=overflow`) If Task B enters deep recursion (e.g., requesting 4 KB of stack space while only 1 KB is available), its stack pointer pushes past its lower boundary.

```
B: recursing
MemManage: CFSR=0x92
  address 0x20000fdc
  cause: data access
  cause: stacking
```

- **The Address (`0x20000fdc`):** This address sits just *below* Task B's partition, plunging into Task A's territory. Since Region 1 only covers Task B, the MPU blocks the write (**DACCVIOL**).
- **The Stacking Error (MSTKERR):** When the fault occurs, the CPU attempts to save its 8-word state frame onto the active stack (PSP). Because the PSP is already out of bounds, this save operation is also blocked.

This perfectly demonstrates why Unit 2 placed the stack at the **bottom** of the partition: the moment the stack overflows, it leaves the allowed memory region, immediately triggering a fault. Without the MPU, this overflow would silently crush Task A's data, eventually causing an incomprehensible crash far from the original error.

1.5.10 Why the Kernel Must Validate Pointers (`DEMO=badptr`)

The MPU **does not** stop `DEMO=badptr`. When a task executes a system call like `sys_write`, the CPU switches to **privileged mode** inside the `SVC_Handler`. Because the kernel has full access to the memory map, it can read or write anywhere. If a task passes a pointer to kernel-private memory, the kernel will blindly access it on the task's behalf, inadvertently leaking or corrupting data.

To prevent this, the kernel must manually verify that every pointer provided by a task points **entirely** to memory that the task itself is allowed to access. For `sys_write`, the buffer `[buf, buf + len)` must fall strictly inside either:

- The caller's **RAM partition**.
- The **FLASH memory** (`0x00000000`–`0x00400000`), where string constants reside.

The Danger of Integer Overflow To validate the buffer, the kernel uses the helper `in_range(p, len, start, end)`. A naive check like `p + len <= end` is a severe security flaw: if `p = 0x20001F00` and `len = 0xFFFFFF00`, the sum wraps around modulo 2^{32} to `0x20001E00`. This passes the naive check, allowing the task to trick the kernel into reading almost all of RAM.

The secure implementation prevents wrap-around by isolating `len`:

```
static int in_range( uint32_t p, uint32_t len, uint32_t start, uint32_t e
{
    /* If p <= end is true, end - p cannot wrap */
    return p >= start && p <= end && len <= end - p;
}
```

Implementing the Check In `sys_write_impl`, the kernel inspects the caller using `task_current()` (which returns `0` if the trusted kernel itself made the call):

```
/* Accept the buffer only inside the caller's partition or inside
 * FLASH (string constants). The kernel itself (no task) is trusted.
const struct task *t = task_current();
if ( t != 0
     && !in_range( a0, len, t->part_start, t->part_end )
     && !in_range( a0, len, 0x00000000, 0x00400000 ) )
    return ( uint32_t ) -1;
```

The Result Running `make run DEMO=badptr` now yields:

```
B: sys_write returned -1
B: uptime 4000 ms (second 1)
```

Because `0x20002004` (the kernel's uptime variable) is neither in B's partition nor in FLASH, `sys_write` immediately returns `-1` before `uart_write` is even called. No kernel data is leaked.

Task B continues running safely because a refused system call is treated as a standard error that the task can handle, not a hardware fault.

The Takeaway: The **MPU** stops unauthorized actions a task attempts directly. **Kernel pointer checks** stop unauthorized actions a task asks the kernel to perform. Both layers of security are strictly necessary.

1.5.11 How a Task Ends: `task_exit_current`

When a task finishes and triggers `SYS_EXIT` (via `svc #4`), the dispatcher calls `task_exit_current()`. This function must decide the next step: launch the next available task, or, if no tasks remain, safely shut down the OS and return to `main()`.

```
uint32_t task_exit_current( void )
{
    kprintf( "task %s exited\n", tasks[current].name );

    /* If there is another task in the list, start it */
    if ( current + 1 < ( int ) NUM_TASKS )
        return task_launch( current + 1 );

    /* Otherwise, no tasks are left. Return to main() */
    current = -1;

    /* asm block: CONTROL = CONTROL & ~1;  (Clear nPRIV to regain privile
    return EXC_RETURN_THREAD_MSP;           /* 0xFFFFFFF9: Thread mode, MS
}
```

The execution flow handles two distinct scenarios:

1. Switching to the Next Task (e.g., Task A → Task B) When Task A ends, it naturally returns to the `task_exit` stub, which triggers `sys_exit`. The dispatcher catches this and executes `task_exit_current()`.

- The kernel calls `task_launch(1)` to prepare Task B.
- **The MPU Shift:** Inside `task_launch`, the kernel updates the MPU. Region 1 is instantly shifted to cover Task B's partition (`RBAR = 0x20001011`). Task A's partition is stripped of its protection region, making it completely invisible and inaccessible to Task B.
- The handler returns `0xFFFFFFFD`, loading Task B's fake frame and starting its execution on the PSP.

2. The Final Task Exits (Returning to `main`) If the last task in the system were to exit, the kernel must hand control back to `main()`, which has been paused since the OS booted.

- The kernel sets `current = -1` to indicate the OS is idle.
- **Restoring Privileges:** It clears the `nPRIV` bit in the `CONTROL` register. This is absolutely essential: `main()` runs in Thread Mode, and if it remained unprivileged, it would instantly trigger a Hardware Fault trying to access the UART to print its final message.
- The handler returns `0xFFFFFFF9`, switching the CPU back to the Main Stack Pointer (**MSP**) exactly where `main()` left off.

Note: In the current Unit, you never actually see `main()` print its final success message because Task B contains an infinite loop and never exits.

1.5.12 End of Unit 3

For the tasks it runs, the kernel now does all four jobs of the week:

- **Placement:** each task has a fixed 4 KB partition built by the linker (stack at the bottom, variables above); code and constants are shared in FLASH.
- **Separate stacks:** tasks run unprivileged on the PSP inside their partition, the kernel on the MSP; `task_launch` enters a task by an exception return and `task_exit_current` ends it and starts the next.
- **Protection by the hardware:** the MPU allows a task FLASH (read and execute) and its own partition (read/write, no execute) and nothing else; a refused access raises MemManage, whose report gives cause and exact address (`rogue`: `0x82`; `overflow`: `0x92`).
- **Protection by the kernel:** `sys_write` refuses any buffer the calling task could not read by itself (`badptr` gets -1), with an overflow-safe `in_range`.

A correct program prints exactly the same output as in Unit 1; what has changed is what a **wrong** one can do.

1.6 Where to Find the Code

The files changed this week, all in `week-04/solution/` (`base/` is the starting point; if these notes and the code disagree, the code is the reference):

- `scripts/mps2_m3.ld`: the two partitions and their `ASSERT`s; `include/cmsdk.h`: MPU registers, field macros, `MMFAR`, CFSR masks.
- `kernel/task.h`, `task.c`: `struct task`, `partition_init`, `task_current`, `task_launch`, `task_start_first`, `task_exit_current`.
- `kernel/svc_entry.s`: `SVC_Handler`; `kernel/syscall.c`: `svc_dispatch`, `in_range`, the checked `sys_write_impl`.
- `kernel/mpu.c`: `mpu_init`, `mpu_set_task_region`; `kernel/faults.c`: `MemManage_Handler`.
- `lib/syscalls.s`, `syscalls.h`: `sys_exit`, `task_exit`, service numbers 4 and 5; `src/main.c`, `task_a.c`, `task_b.c`; `Makefile`: the `DEMO` switch.

1.7 A prova di coglione

1.7.1 Unità 1

I compiti del Sistema Operativo La memoria è una risorsa condivisa. Un sistema operativo completo deve gestire quattro aspetti fondamentali:

- **Collocazione (Placement):** assegnare indirizzi distinti a codice, dati e stack di ogni programma per evitare sovrapposizioni e corruzione dei dati.
- **Condivisione (Sharing):** permettere l'uso comune e sicuro di risorse (es. librerie, buffer hardware, aree di scambio dati tra processi).
- **Protezione (Protection):** impedire l'accesso accidentale o intenzionale alla memoria di altri task.
- **Espansione (Memoria Virtuale):** simulare una quantità di memoria superiore alla RAM fisica, appoggiandosi a memorie di massa.

Il codice di partenza del corso implementa esclusivamente una collocazione di base, e in modo parziale.

Limiti del codice di partenza Ispezionando i binari compilati, si nota che il linker alloca le variabili del kernel e dei task utente una dopo l'altra (regola generica `*(.bss*)`), senza alcun confine strutturale.

- **Collocazione per variabile:** non esiste separazione tra i processi. Se un task calcola un puntatore errato, può sovrascrivere i dati di un altro task in totale trasparenza.

- **Nessuna protezione hardware:** sebbene il sistema separi l'esecuzione temporale (un task inizia dopo la fine dell'altro) e usi una modalità non privilegiata per l'utente, quest'ultima blocca solo istruzioni critiche (es. registri di sistema). Non blocca i normali load/store in RAM. Di conseguenza, un **task malevolo** può accedere e alterare le variabili del kernel (es. il clock di sistema) causando malfunzionamenti globali silenziosi.

Allocazione Statica vs Dinamica La gestione degli indirizzi varia in base al momento in cui vengono assegnati:

- **Statica (Link time):** dimensioni e indirizzi sono fissati prima dell'esecuzione. Non genera frammentazione ed è intrinsecamente prevedibile. È l'unica scelta adatta ai sistemi real-time rigidi (usata in questo corso).

- **Dinamica (Run time):** gestita via *heap* (es. `malloc`). È flessibile ma problematica in ambito embedded: genera **frammentazione esterna** (la memoria libera si divide in blocchi non contigui impedendo nuove allocazioni), i tempi di calcolo sono variabili e i fallimenti avvengono a run time.

Lo spettro della Gestione della Memoria L'evoluzione della gestione della memoria si basa su due pilastri hardware: la **traduzione** (alterazione dell'indirizzo in transito) e la **protezione** (blocco degli accessi).

1. **Flat:** RAM condivisa, zero traduzione e zero protezione (stato iniziale del corso).

2. **Partizioni statiche:** RAM pre-divisa per task al link time (migliore collocazione, nessuna protezione hardware).

3. **MPU (Memory Protection Unit):** protezione hardware pura senza traduzione.

4. **MMU (Memory Management Unit):** paginazione, che offre traduzione hardware e protezione simultanea.

5. **Memoria Virtuale:** paginazione supportata da storage esterno (disco).

Paginazione, MMU e Traduzione a Due Livelli Nei sistemi operativi complessi (es. Linux), i programmi operano su **Indirizzi Virtuali (VA)**, mentre l'hardware accede agli **Indirizzi Fisici (PA)**. Lo spazio è diviso in **Pagine** virtuali e **Frame** fisici di dimensioni identiche (es. 4 KiB).

La **MMU** è il modulo hardware che traduce istantaneamente i VA in PA, scomponendo l'indirizzo: il numero della pagina viene tradotto nel corrispondente frame consultando una *Page Table*, mentre l'offset (l'esatta posizione all'interno della pagina) rimane inalterato.

Per ragioni di efficienza di spazio, non si usa un'unica gigantesca Page Table piatta (che occuperebbe svariati megabyte per processo anche per programmi minimi), ma una **traduzione a due livelli**. Una *Page Directory* principale punta a tabelle di secondo livello che vengono create in RAM *solo* per i segmenti di memoria effettivamente utilizzati dal processo.

TLB (Translation Lookaside Buffer) e Costi Consultare tabelle in RAM per tradurre ogni singolo accesso renderebbe il sistema inusabile. La MMU integra quindi il **TLB**, una cache hardware ad altissima velocità che memorizza le traduzioni virtuale-fisico più recenti.

- **Hit e Miss:** grazie al principio di *località* (i programmi lavorano su porzioni di memoria concentrate come cicli array o stack), la quasi totalità degli accessi trova la traduzione nel TLB (Hit). In caso di assenza (Miss), la MMU deve leggere le tabelle in RAM, moltiplicando i tempi di esecuzione.
- **Jitter da Context Switch:** cambiando processo, il sistema operativo deve invalidare (flush) il TLB per evitare accessi illeciti a frame del processo precedente. I primi accessi del nuovo processo saranno lenti (Miss), introducendo fluttuazioni nei tempi di esecuzione (*jitter*), inaccettabili in sistemi real-time critici.

Page Fault e Memoria Virtuale Se la MMU non trova una mappatura valida o rileva una violazione dei permessi (es. scrittura su memoria in sola lettura), genera un'eccezione hardware detta **Page Fault**.

- Se l'accesso è illecito, il processo viene abbattuto.
- Se l'accesso è valido ma la pagina si trova sul disco, l'OS sospende il processo, trasferisce la pagina in un frame libero della RAM, aggiorna le tabelle hardware e **riesegue l'istruzione interrotta** in modo del tutto trasparente.

La memoria virtuale offre un **isolamento ermetico** tra i task e la capacità di usare enormi quantità di memoria appoggiandosi al disco. Il compromesso è una temporanea latenza catastrofica (misurata in millisecondi anziché nanosecondi) incompatibile con lo sviluppo di sistemi microcontrollori embedded.

MPU vs MMU nei Microcontrollori A differenza dei processori applicativi (Cortex-A), i microcontrollori (Cortex-M) sono sprovvisti di MMU e disco. Adottano invece una **MPU**.

- **Funzionamento:** la MPU non effettua traduzioni. Lavora su indirizzi fisici assoluti controllando se il task in esecuzione ha i permessi (lettura, scrittura, esecuzione) per accedere alla specifica regione di memoria richiesta.
- **Vantaggi:** richiede hardware minimale (pochi registri invece di walker di tabelle e cache), ha bassi consumi e, soprattutto, offre una **latenza rigorosamente deterministica** in quanto priva della variabilità dei TLB Miss.

Responsabilità del Kernel (Il problema del puntatore errato) L'attivazione della MPU limita le operazioni utente, ma non rende il sistema invulnerabile. Se un processo utente passa un puntatore che punta alla memoria privata del kernel a una *System Call*, l'operazione andrà a buon fine. Questo accade perché la system call viene eseguita in **modalità privilegiata**, la quale bypassa le restrizioni MPU imposte al task. **Lezione fondamentale:** l'hardware blocca le esecuzioni utente malevole dirette, ma spetta *esclusivamente* al kernel il dovere di validare sistematicamente i puntatori e i parametri forniti dall'utente prima di processarli.

1.7.2 Unità 2

Nell'Unità 1 i task condividevano la RAM e il task *rogue* poteva corrompere una variabile del kernel. L'Unità 2 fa il primo passo verso la soluzione: dare a ogni task una propria fetta di RAM, la **partizione**, e preparare tutto ciò che serve perché giri lì dentro. È *organizzazione* della memoria, non ancora *protezione*.

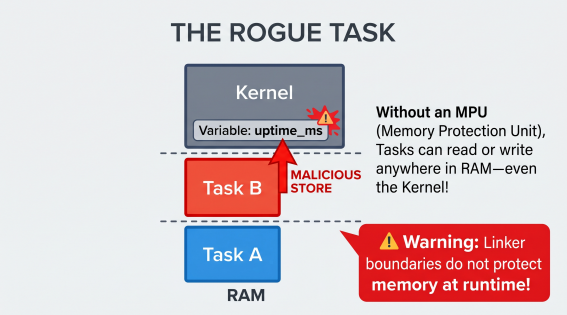


Figure 1.3: Importanza di proteggere la memoria -> accesso non autorizzato

L'idea: una fetta di RAM per ogni task Una partizione contiene solo stack e variabili di un task; la FLASH conserva codice, dati in sola lettura e valori iniziali delle variabili. Cinque regole, ognuna con la sua ragione:

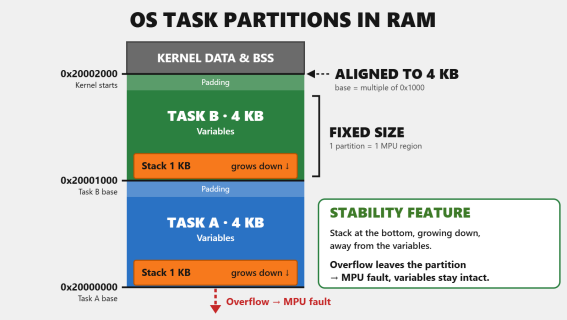


Figure 1.4: Come la memoria viene divisa staticamente (IMPORTANTE: ancora vulnerabile a run-time)

- **Fisse**: le decide il linker (allocazione statica dell'Unità 1); nessun task viene creato, ingrandito o spostato a run time.
- **4 KB allineate a 4 KB**: dimensione potenza di due e inizio multiplo di 4096 sono la forma che la MPU dell'Unità 3 sa proteggere; il prezzo è lo spreco (un task da 1,1 KB occupa comunque 4 KB).
- **Solo RAM**: codice e dati read-only restano in FLASH, *condivisi* da kernel e task (una sola copia di `printf` per tutti).
- **Stack in fondo**: lo stack cresce verso il basso, quindi un overflow esce subito dalla partizione e l'Unità 3 potrà farlo scattare come fault; in cima, invece, sovrascriverebbe in silenzio le variabili del task, e nessuna protezione hardware distinguerebbe quella scrittura da una legittima.

- **Al kernel il resto**: i suoi `.data` e `.bss` seguono le partizioni e il suo stack resta in cima alla RAM.

Il linker costruisce le partizioni Il linker usa pochi concetti. Ogni sezione ha un indirizzo di **esecuzione** (VMA), dove il codice la usa, e uno di **caricamento** (LMA), dove i suoi byte sono salvati nell'immagine. Per i dati differiscono: le variabili *vivono* in RAM, ma i valori iniziali sono *salvati* in FLASH, che li conserva senza alimentazione (`> RAM AT > FLASH`, come per il `.data` della Settimana 2), e qualcuno deve copiarli: il `Reset_Handler` per il kernel, il kernel stesso per le partizioni.

`NOLOAD` riserva indirizzi senza salvare nulla (stack e padding); `ALIGN` arrotonda per eccesso a un multiplo di 4 KB; assegnare il *location counter* permette di saltare a fine partizione; un pattern come `*task_a.o(...)` preleva le variabili di un solo file oggetto, senza marcare il sorgente. Anche `.bss` entra nella stessa sezione dei dati, così i suoi zeri sono in FLASH e un solo ciclo di copia inizializza tutto. Vale infine il **primo match**: le regole delle partizioni vanno prima di quelle generiche del kernel, altrimenti le variabili dei task resterebbero nella RAM del kernel.

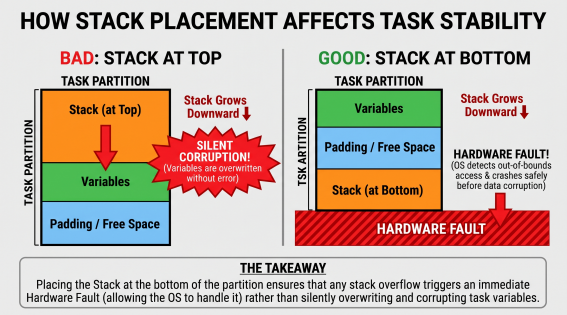


Figure 1.5: Importanza di uno stack alla fine di ogni partizione -> Silent Corruption

Nello script si definisce la dimensione della partizione, si elimina la vecchia sezione piatta degli stack e si inserisce, prima di `.data`, il blocco di A seguito da quello di B. Ogni blocco ha tre sezioni: **stack** (parte dal primo confine da 4 KB, riserva 1 KB e ne segna la cima), **dati** (le variabili del file del task, con estremi allineati alla word perché la copia procede per word da 32 bit) e **padding** (porta il location counter a fine partizione, così la successiva parte dopo i 4 KB completi). Per A l'allineamento parte dall'inizio della RAM, perché dopo `.text` il location counter è ancora in FLASH. I sei simboli per task già usati dal kernel vengono solo *ri-puntati*, e due `ASSERT` fanno fallire il link se i dati di un task non stanno nella sua partizione. Con `nm` si verifica il risultato: due blocchi consecutivi da 4 KB allineati, con lo stack nel primo KB, e i dati del kernel spostati dopo di essi. La cima dello stack coincide con l'indirizzo della prima variabile senza sovrapporsi: è un limite *esclusivo*, perché lo stack del Cortex-M è *full descending* (un push prima decrementa `sp`, poi scrive). Il map file conferma il primo match.

Inizializzare le variabili dei task Il linker decide dove stanno le variabili, ma non dà loro un valore: girano in RAM e nascono in FLASH. Per questo `partition_init` le copia, una word alla volta, prima della prima esecuzione di ogni task: è lo stesso ciclo con cui il `Reset_Handler` inizializza i dati del kernel, lanciato da `task_launch`

una volta per task. Una `struct task` raccoglie ciò che il kernel sa di ogni task (nome, funzione d'ingresso, stack, confini della partizione, indirizzi dei dati) ed è riempita con i simboli del linker, dichiarati come array perché conta solo il loro indirizzo. Per `.bss` non serve azzerare nulla: gli zeri sono già nella sezione caricata. Nella versione di base il ciclo non copiava niente, così il codice preparato girava subito.

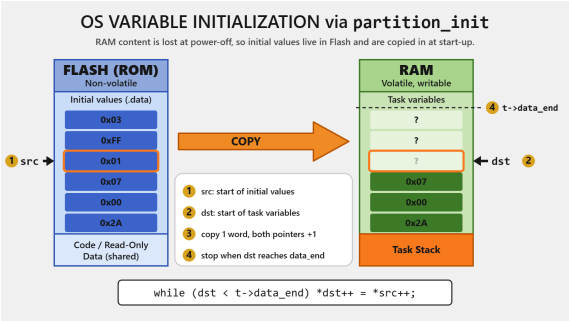


Figure 1.6: Partition Init

Due stack, due livelli di privilegio Il passo successivo è separare stack e privilegi. Il Cortex-M3 ha due stack pointer, e `sp` è quello in uso. **MSP** (main) è sempre usato in *handler mode*, e in thread mode dopo il reset: è lo stack del kernel, in cima alla RAM. **PSP** (process) è usato in thread mode quando il bit SPSEL di CONTROL vale 1: ogni task ha il suo stack PSP *dentro* la propria partizione. Se nPRIV vale 1 il thread mode è non privilegiato, quindi un task gira con **CONTROL = 3**. Lo stack deve stare nella partizione perché contiene variabili locali, indirizzi di ritorno e frame delle eccezioni: altrimenti non si potrebbe confinare il task nei suoi 4 KB senza togliergli lo stack.

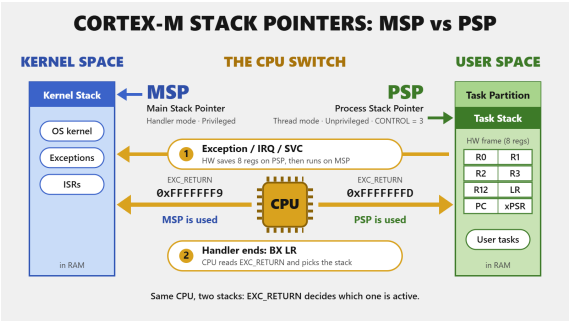


Figure 1.7: MSP & PSP - Exception Return

E poiché l'handler mode usa sempre MSP, il lavoro del kernel non passa mai dallo stack di un task. Il ponte tra i due mondi è **EXC_RETURN**, il valore in `lr` che chiude un'eccezione: `0xFFFFFFF9` torna in thread mode su MSP, `0xFFFFFFFD` su PSP. All'ingresso in un'eccezione l'hardware ne mette uno in `lr`, e il suo bit 2 indica dove sta il frame salvato (1 = PSP, 0 = MSP).

Avviare un task: il frame finto Resta da far partire un task. Il kernel opera in handler mode, su MSP e con tutti i privilegi: una normale chiamata di funzione farebbe ereditare questo stato al task, e un suo bug potrebbe corrompere il kernel.

Il task deve invece girare in thread mode, non privilegiato e su PSP. Ma nessuna istruzione cambia insieme stack, privilegi, modalità e indirizzo: solo l'**exception return** compie questo "triplo salto" in modo sicuro e atomico. Un task appena creato, però, non è mai stato interrotto: il kernel *finge* allora il passato, costruendo sul suo stack un frame di 8 word (32 byte) identico a quello che l'hardware si aspetta, puntando il PSP a quel frame e ordinando il ritorno con `0xFFFFFFFD`. La CPU non si accorge dell'inganno: ripristina i registri, abbassa i privilegi, passa in thread mode e salta all'indirizzo iniziale del task.

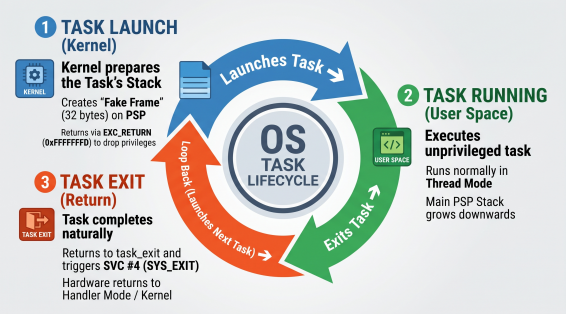


Figure 1.8: Ciclo di vita di un task (gestito dal OS)

La sequenza parte da `main` con la system call di avvio; l'handler intercetta la chiamata e fa partire `task_launch`, che compie tre operazioni. Prima la *preparazione*: registra quale task parte, predispone la MPU a proteggerne la partizione e chiama `partition_init`. Poi costruisce il *frame* sotto la cima dello stack: `r0-r3` e `r12` a zero, `lr` = `task_exit` (se la funzione del task ritorna, il task esce in modo pulito invece di andare in crash), `pc` = punto d'ingresso, `xPSR` con il bit Thumb, obbligatorio sul Cortex-M (senza, scatta una `UsageFault`). Infine il *cambio di contesto*, in assembly: carica il frame nel PSP, attiva nPRIV in CONTROL e forza il ritorno di `0xFFFFFFFD`. L'hardware estrae i 32 byte e il task parte non privilegiato, con il PSP, liberato dal frame, esattamente in cima allo stack.

System call e fine di un task Ora una `svc` può arrivare da due ambienti: da `main`, una sola volta per avviare il sistema (su MSP), e dai task, per le chiamate ordinarie (su PSP). L'`SVC_Handler` deve quindi scoprire dove sta il frame, e lo fa leggendo il bit 2 dell'EXC_RETURN in `lr`: 0 = MSP, 1 = PSP. In assembly sceglie lo stack giusto e chiama la funzione C `svc_dispatch`, passandole frame ed EXC_RETURN; il valore restituito è un nuovo EXC_RETURN, con cui l'assembly esce dall'eccezione. Prima l'handler tornava sempre al punto di chiamata; ora è `svc_dispatch` a decidere *dove tornare*. Con `SYS_START_TASKS` restituisce il codice per lanciare il primo task (`task_start_first`); con `SYS_EXIT` quello per lanciare il successivo o tornare a `main` (`task_exit_current`); per le chiamate normali scrive il risultato in `frame[0]` (che diventa `r0` del task) e restituisce l'EXC_RETURN originale, così l'esecuzione riprende da dove si era interrotta. È la regola d'oro: **il kernel non aspetta mai dentro un handler**; ogni system call finisce subito e guida la CPU verso ciò che deve girare dopo.

Un task può terminare chiamando `sys_exit()`, uno stub che non ritorna mai, oppure semplicemente ritornando dalla sua funzione: atterra così in `task_exit`, l'indirizzo che `task_launch` ha messo nel `lr` del frame finto, che a sua volta chiama

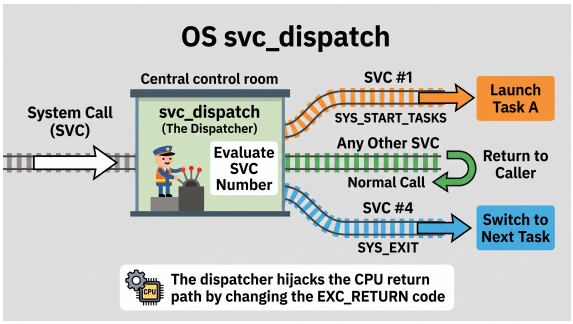


Figure 1.9: Opzioni di un dispatcher

`sys_exit`. Il Task A usa questa seconda strada. Il frame finto, quindi, non solo avvia il task ma ne prepara anche l'uscita; il lato kernel si vedrà nell'Unità 3.

Verifiche e limite delle partizioni Con `gdb` si controlla la catena: in entrambi i task **CONTROL = 3**; il **PSP** coincide con la cima dello stack, dentro la partizione del task; il frame mostra `lr` = `task_exit` (più il bit Thumb), `pc` = funzione del task e `xPSR` con il bit Thumb; l'**MSP** è lo stesso nei due casi, perché contiene il frame di `main` in attesa e i task non lo toccano mai.

Eppure, rieseguendo la demo *rogue*, il Task B scrive ancora nella variabile del kernel (ora spostata, fuori dalla sua partizione) e l'orologio viene corrotto *esattamente come nell'Unità 1*. Il linker decide solo *dove* stanno le cose: una scrittura è una scrittura ordinaria e, a run time, nulla la confronta con i confini della partizione. Le partizioni *dispongono* la memoria ma non *impongono* nulla. È cambiata però la forma: la memoria di ogni task è ora un blocco unico da 4 KB allineato, quindi basta *una* regola hardware per task per descrivere ciò che può toccare. Quella regola è la MPU, oggetto dell'Unità 3.

In breve Partizioni fisse da 4 KB allineate, con lo stack in fondo e il codice condiviso in FLASH → costruite dal linker (`ALIGN`, `NOLOAD`, `AT > FLASH`, pattern per file, primo match, `ASSERT`) → riempite da `partition_init` → eseguite in modo non privilegiato su PSP, con il kernel su MSP → avviate da `task_launch` con un frame finto e `0xFFFFFFFD` → gestite nelle system call da `SVC_Handler` e `svc_dispatch` → chiuse da `task_exit`. La memoria è ora *organizzata*, ma non ancora *difesa*: per farla rispettare serve la MPU dell'Unità 3.

1.7.3 Unità 3

L'obiettivo è impedire **via hardware** che un task esca dai propri confini. L'Unità 1 aveva introdotto la MPU come lista di regioni controllate a ogni accesso e l'Unità 2 aveva preparato la memoria (una partizione da 4 KB, allineata, per ogni task): ora si programma la MPU perché ogni task veda solo la FLASH e la propria partizione, e si osserva cosa accade quando prova a uscirne.

Come ragiona la MPU. Il Cortex-M3 ha **8 regioni**; ognuna ha **base**, **dimensione** e **permessi** e vale solo se il suo bit ENABLE è attivo. La dimensione è una potenza di 2 tra 32 byte e 4 GB e la base deve essere un **multiplo della dimensione**: per questo l'Unità 2 ha reso le partizioni di 4 KB e allineate a 4 KB, così che

una regione copra esattamente una partizione. Il controllo vale per il codice **non privilegiato** (i task): un accesso fuori da ogni regione attiva, o vietato dalla regione, viene **bloccato** e la CPU solleva un fault MemManage. Per il codice **privilegiato** (il kernel), il bit PRIVDEFENA decide se, dove nessuna regione corrisponde, valga la stessa regola o la mappa di memoria predefinita della scheda. La MPU è una periferica del core scrivibile solo in modo privilegiato, quindi un task non può spegnerla.

I permessi. Il campo **AP** (3 bit) assegna diritti distinti al codice privilegiato e a quello non privilegiato: qui servono lettura/scrittura per entrambi sulla partizione del task in esecuzione e sola lettura sulla FLASH. Il bit **XN** (*execute never*) vieta il *fetch* di istruzioni, mentre letture e scritture seguono AP: le partizioni contengono solo dati e stack, quindi sono XN, e un task non può eseguire codice che ha scritto lui stesso in RAM, chiudendo un attacco classico. Con **PRIVDEFENA**=1 il kernel conserva l'accesso a tutta la RAM e alle periferiche senza una regione per ciascuna (con 0 sarebbe limitato anche lui): si sceglie 1 perché il kernel è fidato e lo scopo della MPU è *contenere i task*.

Registri e calcolo dei campi. Si usano cinque registri: `MPU_TYPE` (numero di regioni, sola lettura), `MPU_CTRL` (ENABLE e PRIVDEFENA), `MPU_RNR` (regione corrente), `MPU_RBAR` (base) e `MPU_RASR` (XN, AP, S/C/B, SIZE, ENABLE). C'è una sola coppia RBAR/RASR per tutte le regioni e RNR sceglie a quale si riferisce; scrivendo RBAR con il bit VALID si imposta anche il numero di regione, e una scrittura ne sostituisce due. Il campo SIZE contiene $\log_2(\text{dimensione}) - 1$ (il -1 fa arrivare 5 bit fino a 4 GB): vale 11 per 4 KB e 21 per 4 MB. S, C e B (tipo di memoria per cache e multi-core) qui non contano, perché non c'è cache; TEX e SRD restano a 0. Gli esercizi fissano le regole: una regione da 32 KB ha SIZE=14 e parte da un multiplo di 32 KB (non di 4 KB); un task da 5 KB richiede 8 KB, la potenza di 2 successiva, con 3 KB sprecati come nelle partizioni dell'Unità 2.

Le due regioni di questa unità. Il disegno è volutamente minimo. La **Regione 0** copre la FLASH (4 MB, sola lettura, eseguibile) ed è uguale per tutti i task: codice e costanti stringa. La **Regione 1** copre la partizione del task in esecuzione *adesso* (lettura/scrittura, XN) e il kernel la sposta a ogni avvio di task. Tutto il resto (l'altra partizione, dati e stack del kernel, le periferiche) non ha regione: per un task ogni accesso lì è un fault. Lo "stato MPU" di un task si riduce così a un solo valore, la base della Regione 1.

Programmare la MPU. Due funzioni, già richiamate nei punti giusti ma vuote nella base (MPU spenta):

- `mpu_init`, chiamata una volta da `main` prima di avviare i task, programma la Regione 0 e abilita la MPU (ENABLE e PRIVDEFENA); dopo il reset la MPU è spenta e nessuna regola vale finché ENABLE non è impostato. La Regione 1 non c'è ancora, ma non è un problema: `main` e il kernel sono privilegiati e, grazie a PRIVDEFENA, usano la mappa predefinita, e nessun task parte prima che `task_launch` la imposti.

- `mpu_set_task_region`, chiamata da `task_launch` prima di entrare in ogni task, riceve base e dimensione della partizione, ne ricava SIZE (il logaritmo di una potenza di 2 è il numero dei suoi zeri finali, che GCC conta con `__builtin_ctz`) e sposta la Regione 1 (lettura/scrittura, XN); il linker script garantisce potenza di 2 e allineamento.

Dopo le scritture ai registri MPU si usano `dsb` (attende che siano concluse) e `isb` (le istruzioni successive vedono le nuove impostazioni), così nessun accesso è controllato con le vecchie regioni; con le macro di `cmsdk.h` il codice resta leggibile e `make` compila senza warning.

Quando la MPU dice no: il MemManage fault. Un accesso rifiutato **non avviene** (una store non scrive, una load non legge) e la CPU solleva MemManage, l'eccezione 4, abilitata al boot con MEMFAULTENA in SHCSR (Settimana 2) e servita da `MemManage_Handler`, che gira privilegiato sull'MSP. Il byte basso di **CFSR** ne indica la causa: `IACCVIOL` (fetch rifiutato, ad esempio da una regione XN), `DACCVIOL` (load o store rifiutata), `MSTKERR` (salvataggio del frame di 8 word all'ingresso di un'eccezione rifiutato), `MUNSTKERR` (ripristino all'uscita rifiutato) e `MMARVALID`, che segnala che **MMFAR** contiene l'indirizzo rifiutato. MMFAR va letto solo se MMARVALID vale 1: con un fault di fetch o di stacking la CPU non lo scrive e può contenere un valore vecchio. L'handler legge CFSR una sola volta, stampa l'indirizzo solo se valido e controlla ogni causa con un `if` separato, perché più bit possono essere attivi insieme (`kprintf` funziona perché l'handler è privilegiato).

La protezione in azione. Con la MPU attiva, tre scenari:

- **Esecuzione normale** (`make run`). Output identico a quello dell'Unità 1, ma sotto controllo hardware: A e B usano solo FLASH e la propria partizione. Nelle system call la CPU passa in Handler mode, dove il kernel, privilegiato, non è vincolato dalle regioni dei task; al cambio da A a B si aggiorna solo la Regione 1.

- **Task "rogue"** (`DEMO=rogue`). B tenta di sovrascrivere una variabile del kernel (il contatore dell'uptime): la store non viene mai eseguita e il MemManage riporta DACCVIOL con MMARVALID e l'indirizzo esatto della variabile. L'orologio resta intatto e il bug è colto proprio all'istruzione colpevole.

- **Stack overflow** (`DEMO=overflow`). B va in ricorsione profonda (4 KB di stack richiesti, 1 KB disponibile) e supera il limite inferiore dello stack: l'indirizzo guasto cade appena sotto la sua partizione, nel territorio di A e fuori dalla Regione 1 (DACCVIOL). Compare anche MSTKERR: per gestire il fault la CPU deve salvare il frame sullo stack attivo (PSP), già fuori limite, e anche questo salvataggio viene bloccato. Ecco perché l'Unità 2 aveva messo lo stack *in fondo* alla partizione: appena trabocca, esce dalla regione consentita e scatta il fault, mentre senza MPU avrebbe corrotto in silenzio i dati di A, con un crash lontano dall'errore originario.

Ciò che la MPU non vede: i puntatori passati al kernel. Il caso `badptr` sfugge alla MPU. In una system call come `sys_write` la CPU è privilegiata dentro

`SVC_Handler` e, proprio perché il kernel è fidato, può accedere ovunque: se un task gli passa un puntatore a memoria privata del kernel, il kernel lo segue ciecamente per suo conto, rivelando o corrompendo dati. Deve quindi verificare che ogni puntatore punti *interamente* a memoria accessibile al task: per `sys_write`, il buffer deve stare nella partizione RAM del chiamante o nella FLASH (dove stanno le stringhe costanti). Il controllo `in_range` va scritto con cura: il test ingenuo "base più lunghezza $\leq$ fine" è una falla, perché la somma può andare in wrap-around modulo 2^{32} e risultare piccola, facendo passare una lunghezza enorme e lasciando leggere quasi tutta la RAM. La versione sicura controlla che la base sia nell'intervallo e poi confronta la lunghezza con lo spazio rimasto, senza mai sommare. In `sys_write_impl` il kernel identifica il chiamante con `task_current()` (0 se chiama il kernel stesso, fidato) e rifiuta i buffer fuori dalle due zone: con `DEMO=badptr`, `sys_write` restituisce -1 prima di toccare l'UART, nessun dato esce e B prosegue, perché una syscall rifiutata è un normale errore gestibile dal task, non un fault hardware. In sintesi, la **MPU** ferma ciò che un task fa *direttamente*, i **controlli del kernel** ciò che un task *chiede al kernel* di fare: servono entrambi.

La fine di un task. Al termine, un task torna a uno stub che chiama `SYS_EXIT`; il dispatcher invoca `task_exit_current`, che avvia il task successivo o, se non ce ne sono, riporta l'OS a `main`. Da A a B, `task_launch` sposta la Regione 1 sulla partizione di B (quella di A diventa inaccessibile) e il ritorno dall'eccezione carica il frame finto preparato per B, avviandolo sul PSP. Se terminasse l'ultimo task, il kernel segna che non ce ne sono più, azzera `nPRIV` in `CONTROL` per riprendere i privilegi e ritorna in Thread mode sull'MSP, riprendendo `main` dove si era fermato: è essenziale, perché `main` gira in Thread mode e, se restasse non privilegiato, causerebbe un fault già al primo accesso all'UART per il messaggio finale (che qui non si vede mai, perché B non termina).

Il quadro finale. Per i task che esegue, il kernel svolge ora i quattro compiti della settimana:

- **Collocazione:** partizione fissa da 4 KB per task, costruita dal linker (stack in fondo, variabili sopra); codice e costanti condivisi in FLASH.
- **Stack separati:** task non privilegiati sul PSP, nella propria partizione; kernel sull'MSP; `task_launch` entra in un task con un ritorno da eccezione e `task_exit_current` lo conclude e avvia il successivo.
- **Protezione hardware:** la MPU concede a un task solo la FLASH (lettura ed esecuzione) e la propria partizione (lettura/scrittura, senza esecuzione); ogni accesso rifiutato genera un MemManage con causa e indirizzo esatto.
- **Protezione software:** `sys_write` rifiuta ogni buffer che il task non potrebbe leggere da solo, con un `in_range` a prova di overflow.

Un programma corretto stampa esattamente come nell'Unità 1: è cambiato ciò che può fare uno **sbagliato**.